

Required Readings

Required Readings for this program are open access, which means you can access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Consoli, Sergio, et al. *Data Science for Economics and Finance: Methodologies and Applications*. Springer Nature, 2021. <https://link.springer.com/book/10.1007/978-3-030-66891-4#toc>.

- **Required pages:** The entire chapter titled "Data Science Technologies in Economics and Finance: A Gentle Walk-In"
- **Estimated reading time:** 20 minutes

1. Yu, Bin. "Veridical Data Science." *Proceedings of the 13th International Conference on Web Search and Data Mining*. 2020. https://vdsbook.com/01-veridical_ds.

- **Required pages:** Part 1, Chapter 1
- **Estimated reading time:** 25 minutes

1. Consoli, Sergio, et al. *Data Science for Economics and Finance: Methodologies and Applications*. Springer Nature, 2021. <https://link.springer.com/book/10.1007/978-3-030-66891-4#toc>.

- **Required pages:** Section 3 of the chapter titled "*Massive Data Analytics for Macroeconomic Nowcasting*" (Section 3 title: "*Example of Macroeconomic Applications Using Massive Alternative Data*")
- **Estimated reading time:** 20 minutes

FINANCIAL DATA BEST PRACTICES

Reading Time	60 minutes
Prior Knowledge	data characteristics, data types, financial time series
Keywords	data, alternative data, economics, big data, quality, accuracy, timeliness, completeness, consistency, 5Vs, volume, variety, veracity, value velocity

In Financial Markets, we examined how the world of financial engineering is influenced by concepts like credit risk, financing, volatility, correlation, liquidity, regulation, leverage, non-linearity, model failure, and financial crisis. In Financial Data, we will revisit all of these concepts by introducing data that can help us populate models that can interpret, predict, and even prescribe solutions to the challenges financial engineers face.

In this lesson, we will examine what constitutes financial data: where it comes from, the factors that affect data, the types of data, data structures, and data complexities. We will provide an overview of the data sources and data identifiers along with the challenges of combining data from different time zones.

1. The Evolution and Importance of Data in Finance

The explosion of available data in recent years has transformed the way professionals approach financial analysis and decision-making. Traditionally, one would expect that financial analysts would rely on market data, indicators, and corporate financial statements. However, nowadays, an analyst delves into health data, satellite images, climate data, noise levels and pollution data, education data, and more.

Let's think for a moment: are these diverse datasets of any value for financial analysis? Can an investment decision be influenced by this information?

The answer is YES, and it will be proven by real-life examples throughout the Financial Data course. Economics is intertwined with society and any data produced can be seen from an economic perspective. To illustrate:

- Assume an investor is considering an investment in a pharmaceutical company. In addition to the financial reports and market data, health data can provide insight into trends and disease prevalence and, subsequently, to future demand for a company's products.
- Satellite data can shed light on crop health, harvest levels, and water usage, which is quantifiable information for commodity pricing in agricultural business.
- Climate data can be an added value in predicting long-term impact on crop yields and energy demand among others.

This diverse set of data sources is referred to as "alternative data" and is becoming increasingly valuable in financial analysis. The analysis of such datasets is made possible by advancements in technology: the emergence of big data infrastructures, cloud computing, and analytical tools. More specifically, several key developments facilitated the data revolution in finance:

- Digitization of information led to the exponential growth of data availability.
- High-performance computing and distributed systems allowed analysts to handle massive datasets. Platforms and cloud services such as **Apache Kafka**, **Amazon Kinesis**, **Google Cloud Pub/Sub**, and **Microsoft Azure Event Hubs** among others provide essential infrastructure for streaming and processing millions of events per second.
- Advanced analytical tools that can often be accessed freely are pivotal in performing statistical analysis, modeling, and inference. For example:

Free Tools

- Python: an open source programming language that can be used for data analysis and manipulation using libraries such as NumPy, Pandas, SciPy and more.
- R: Another programming language specifically created for statistical computing. It is used in finance for econometric analysis and data visualization, but there are also available libraries that allow for geometrical and topological analysis.
- Jupyter Notebooks: A researcher can execute blocks of code in cells without the need for executing the entire script. This approach allows for continuous testing of specific parts of an analysis without the need to restart the kernel. Of course, one can write markdown text (like this text) and mathematical expressions.

Paid Tools

- Bloomberg Terminal, Thomson Reuters Eikon, and more: These terminals provide real-time analytics and financial data for trading and risk management.
- MATLAB & Mathematica: Numerical environments that provide the required tools for engineers and financial analysts alike.
- SAS, Stata, and EViews: Statistical software packages that are used in business and finance for data analysis, econometric modeling, and forecasting.

- Advancements in protocols led to real-time data consumption and analysis.

- WebSocket Protocol: it utilizes a single, continuous, bidirectional, and persistent connection between the client and the server. Through this always-open channel, the server can broadcast market feeds, such as trades, order book updates, and price changes, in real time to the client. Simultaneously, the client can place or cancel orders instantly without establishing a new connection for each interaction.
- Financial Information Exchange (FIX) Protocol: The FIX protocol is a vendor-agnostic messaging standard that provides a standardized method of electronic communication between financial institutions. Due to its widespread adoption and real-time communication, it has become the industry standard for electronic trading. Originally created in 1992 through a collaboration between Salomon Brothers and Fidelity Investments, the FIX protocol has since undergone several revisions to accommodate various improvements. For more detailed information, you can visit the official FIX Trading Community website <https://www.fixtrading.org/>.
- Precision Time Protocol (PTP): PTP allows for precise time synchronization across networks. Exchanges use this protocol to ensure that trades are accurately time-stamped to the microsecond. The latter is crucial for high-frequency trading and pivotal in ensuring that the **MFID II** timestamp requirements are met (Netnod).

2. Types of Financial Data

Broadly speaking, there are three types of data: structured data, unstructured data, and semi-structured data.

2.1 Structured Data

Typically, structured data is *rectangular* (structured data can be in multidimensional arrays as well but in the context of finance, an analyst will mostly face tabular data). It can be organized into rows and columns. Rows reflect observation times; columns represent variables.

For example, you could have a 1000 by 50 matrix of stock prices. One thousand rows represent approximately 4 years of daily closes (i.e., business days). Fifty columns represent the 50 stocks within an exchange-traded fund. This data can easily be stored thanks to a friendly structure.

- Cross-Section: One row gives you a snapshot of the portfolio at a particular moment in time.
- Time Series: One column gives you a "movie" of a stock over four years.

In this example, the difference between observations is effectively the same: one business day (even though they may range from one calendar day for weekdays to three calendar days for the weekend or even four calendar days for three-day weekends).

It is also possible to have structured data where the observation times form an irregular time series. For example, if these 1000 points reflect intraday observation times, then the time itself could be a variable.

Structured Data in Python

In Python, structured data can be handled by a variety of data structures, but pandas.DataFrame is often the best choice after we have loaded the dataset on memory. However, numpy.array can also be used when dealing with homogeneous datasets and performance is a priority.

Long-Term Storage of Structured Data

For long-term storage though, financial structured data is stored in databases or specialized file formats. The choice of storage depends on the use case:

- Time Series Databases (TSDB) such as InfluxDB, KDB+/q, TimescaleDB, and Prometheus are optimized for storing and querying time-series data.
- Columnar File Formats such as Apache Parquet or Apache ORC, which are commonly used with big data analytics where loading times are a concern.
- Human Readable Files such as CSV (Comma Separated Values) and Excel files are easy to read and share since opening such as file does not require specialized knowledge. These files are not particularly efficient.
- Other Specialized Data Formats such as HDF5 (Hierarchical Data Format version 5), which are excellent choices for large numerical datasets due to their support for complex data hierarchies. These are useful for scientific computing with large numerical datasets.

2.2 Unstructured Data

Most financial data is numeric, but of course there is also non-numerical data. These data types include social media posts, review sites, photographs, audio, and video files. If you are pricing weather derivatives, perhaps satellite photos of weather are considered key data. If you are monitoring particular companies, perhaps the sentiment from Facebook, Twitter, or other social media sites reflect the short-term optimism about the stock. Likewise, you might examine customer reviews on Yelp or Instagram for photographs from satisfied users to determine product adoption, popularity and brand loyalty, and customer churning. Indeed, much of machine learning takes inputs from this type of data and builds models known as sentiment analysis. Unstructured data is not as easy to store as structured data, due to the unconventional, non-rectangular format.

Unstructured Data in Python

In Python, unstructured data can be handled by dictionaries and lists, which can allow for flexible keys, data types, and data lengths. Both dict and list can become as nested as we like and accommodate whatever type of data we need. For example, we can have a set of dictionaries whose schemas are not alike: some values of some dictionaries are lists and inside those lists, we can have some more dictionaries whose values can be images. At the same time, in some other dictionaries, the values are DataFrame objects which hold semi-structured document data!

2.3 Semi-Structured Data

Data forms a spectrum ranging from unstructured to structured. We can consider a category in between. For example, emails are semi-structured: they have well-defined fields such as "To," "From," "Subject," and "Date," but the body of the email is unstructured, consisting of free-form text.

Characteristics of Semi-Structured Data

Semi-structured data include some form of schema, which makes it more searchable and indexable than unstructured data. However, the data do not follow a strict format. A good example would be the JSON and XML files where the schema could change from file to file and contains nested data structures. Typically, in their semi-structured form, many data types will follow a structure.

Semi-Structured Data in Python

In Python, semi-structured data can be handled by a combination of data structures. A very likely approach for handling such data are the dictionaries and lists as explained in the *Unstructured Data* section. If a Python data type can hold unstructured data, then it is evident that it can handle semi-structured and structured data as well.

But we can utilize pandas.DataFrame along with its powerful MultiIndex as well in any case where the schema is such that it can be organized in tabular format.

3. Data Sources

In the previous lesson, we discussed the importance of the vendor.

Our first consideration is the quality of the data the vendor provides.

Informally speaking, there are four types of providers of data:

A. Primary Sources

These include exchanges and credit agencies, like rating agencies, credit cards, and credit bureaus. Exchanges and credit agencies are the locations where the data originates based on transactions, trading, payments, or lack thereof. Here are examples of exchanges and credit institutions:

- Stock Exchanges: NYSE, NASDAQ, Shanghai, Shenzhen, Euronext, Nigerian, Nairobi, Stock Exchange of Mauritius, etc.
- Option Exchanges: Chicago Board Option Exchange
- Commodity Exchanges: Chicago Mercantile Exchange, London Metal Exchange
- Forex Exchanges: London, New York, Sydney, Tokyo
- Cryptocurrency Exchanges: Bitcoin, Coinbase, Binance, Gemini, FTX
- Credit Agencies: S&P, Moody's, Fitch, Transunion, Equifax, Experian, Visa, Mastercard

B. Financial Platforms

These companies provide trading platforms, news, analytics, and communications. While they may provide data, they are typically redistributors of data from exchanges, OTC markets, and other financial sources. The data can be analyzed from inside their platform or through an API.

- Bloomberg
- Thomson-Reuters
- Factset

C. Data Vendors

These companies tend to focus on data and analytics. They tend to have their data accessible through APIs. Their revenue typically comes from selling or streaming data to clients.

- BarChart
- RavenPack
- Capital IQ
- CRISP
- Datastream
- Morningstar
- Exante Data
- Xignite
- Refinitiv
- Short Trends
- dxFeed

D. Free Sites

Finally, there are free sites, whether public services, government sites, or user groups, that pool and provide access to free data for educational purposes or individuals. Some of these limit the amount of data that can be accessed.

- Quandl
- Alpha Vantage
- FRED: Federal Reserve Economic Database
- Yahoo Finance
- Google Finance

The emphasis here is not so much on knowing which vendor is in which category but rather how they may offer the data. Exchanges can often provide complete data, including quote and trade data. A free site like Yahoo Finance may only be able to provide end-of-day data based on trades and not quotes. Some vendors may clean the data; others present the data as they collected it and leave it to customers to address errors. Some data may be in a format known as FIX: Financial Information eXchange Protocol. Other data may simply be in comma-separated values, tab-delimited formats, etc.

Data itself is a commodity, and exchanges, markets, and vendors make money providing this data, whether historical, daily, or real-time, to clients who need to make quick, if not real-time, decisions with it.

4. Data Descriptors and Identifiers

How do you refer to data related to specific securities? Most securities have identifiers. Depending on the asset class, different types of identifiers are used. In Financial Markets, we have already discussed bonds, stocks, cryptocurrencies, ETFs, options and securitized products among others. Let's start with stocks.

Tickers

Tickers are the most popularly known identifiers. You will see tickers listed on screens during television shows and in newspapers and online stories, as the ticker may tend to have some resemblance to the name or type of company. Everyone knows Facebook as FB (even though it is now Meta), Apple as AAPL, and IBM as, well, IBM! The ticker is not only an identifier but a way to remember the company. For example, Papa John's is a food company that specializes in pizza, and its ticker is PZZA. Thermonesis Corporation is an energy company with the ticker KOOL. Southwest Airlines' ticker is LUV. LiveOpenWest has ticker WOKW.

Tickers are important but tend to be applied to stocks and ETFs. Bonds, for example, do not have tickers. Let's look at some other ways securities are identified.

CUSIP

CUSIP: Typically, when you see all uppercase, the identifier is an acronym. CUSIP is an acronym for Committee on Uniform Security Identification Procedures. Used in North America, it is a nine-digit alphanumeric code that uniquely identifies not only a security but also information about its type and issuer. The first 6 alphanumeric characters identify the issuer. The next two characters identify the type of security. The last digit is a check that the code is created correctly. CIN is a CUSIP for securities in foreign markets. These are extra digits, where the first letters represent the issuing country. The acronym refers to the CUSIP International Numbering System.

SEDOL

SEDOL is also an acronym: Stock Exchange Daily Official List. When a company is listed on an exchange, it has a SEDOL. If it gets delisted, it loses its SEDOL. Consider the case of Hertz Rental Cars. Hertz was the number one rental company in the world before it ran into financial difficulties. It continued to suffer during the COVID-19 pandemic, which affected both business and leisure travel. In May 2020, Hertz filed for Chapter 11 bankruptcy. Later that year in October 2020, Hertz was delisted from the NYSE. At that time, the SEDOL changed representing a listing no longer on the NYSE but on NASDAQ. Consequently, any data collected on Hertz must recognize the switch from one exchange to another.

ISIN

ISIN is the International Securities Identification Number. ISIN uses 12-digit identifiers. The first two characters are letters identifying the country. The next 9 alphanumeric characters identify the security. ISINs are more universal. For example, futures tend not to have CUSIPs but do have ISINs. Foreign exchange rates have identifiable tickers (e.g., GBP/USD) but are referred to internationally by their ISINs.

The securities we've described are fungible. Fungible securities are interchangeable. For example, 100 shares of AAPL common stock are like any other 100 shares of AAPL common stock. Most financial securities are fungible. Note that fungible need not mean identical. Gold is considered fungible, so long as it does not have serial numbers or marks that uniquely identify it. (U.S. dollar bills have serial numbers but are still considered fungible.) Likewise, Bitcoin is fungible. You can pay someone a Bitcoin, and they can send you a different one back. They are considered interchangeable, just like paper currency in any country.

When we get to items that are more specific or unique, we have infungible securities. Real estate is not a fungible commodity because it is not interchangeable. Therefore, real estate is infungible. Even two houses that are the same size and material and located in the same neighborhood are not interchangeable in part because they are in different physical locations, with different views, light, neighbors, properties, etc. Similarly, works of art are not fungible. An original painting or sculpture is not like any other. Securities that are infungible will be very difficult to identify because they are, by nature, unique.

For example, Zillow may show a property that was \$100,000, and then a year later, it is \$500,000. Did the house really increase five-fold? No. Zillow represents the value of whatever the real estate is at the address. Originally, the land was undeveloped; it was a vacant lot. It was bought by a builder for \$100,000. The builder constructed a house that was then sold for \$500,000. The land itself has value ("undeveloped") and any structures (homes, buildings) add value through "developments" on the land. Zillow relies on a deed or property record, often filed at the local level, to refer to the property. With millions of properties, it is difficult to uniquely identify a property and likely impossible (and unnecessary) to have an internationally standardized system.

Moving to precious jewelry, diamonds are infungible. In the U.S., some diamonds are graded by the Gemological Institute of America (GIA) and have an engraved serial number. Similar to gold with a serial number, this makes diamonds not interchangeable and thus infungible. Diamonds without this engraving are more fungible. However, there are risks that it is not properly appraised for clarity or carats or that the "gold" is actually man-made or fake.

When you want to analyze the behavior of securities, think about the data you are collecting, and ask yourself if it is a specific, one-of-a-kind asset (home, art, jewelry) or a fungible, easily interchangeable asset (stock, bond, ETF). Use identifiers if they help.

5. Time Zones, Trading Days, and 24/7 Trading

"Get me the closing price" is something you will hear as a financial engineer. But what is "the" closing price?

The answer sounds like a quantum physics one: depending on where you are, the close will be different, based on your location and time zone.

Clearly, if we are in Lagos, Nigeria, trading Dangote Cement, we can expect its official close at the close of the Nigerian Stock Market—2:30 PM local time or GMT+1. If we are in Sydney, Australia, trading the Australian stock BHP, then we observe the market at its closing time, 4:00 PM Sydney time: GMT + 10 hours.

Suppose we want to correlate the returns of these two companies using closing prices. We have to remember that their official exchange closes are about nine hours apart! Here is a table of times the markets open and close:

GMT	Sydney, Australia	Lagos, Nigeria	Event
00:00	10:00	01:00	Sydney Stock Exchange opens
06:00	16:00	07:00	Sydney Stock Exchange closes
09:00	19:00	10:00	Nigeria Stock Exchange opens
13:30	23:30	14:30	Nigeria Stock Exchange closes

Depending on the objective of the analysis, one could use

- Plan A: GMT=16:00. Get the data at Sydney's close, but the Nigerian exchange is not open!
- Plan B: GMT=13:30. Get the data at the Nigerian's close, but the Sydney exchange has already closed.

The advantage of Plan B is that the date will reflect the closing prices of both exchanges. Even though Sydney closed, that is the last observable price for that trading day. Keep in mind that data is not available until at least GMT+13:30 when the Nigerian exchange closes.

If you were to build a trading strategy using this closing correlation and decided to buy Dangote, it would have to be for the following trading day. Indeed, novices will often lose a sense of the timestamp. If you were to build a strategy that buys Dangote based on the correlation, we would reverse cause and effect.

The difficulties are more subtle if we look at the same security that trades in different parts of the world. Consider a credit derivative that was discussed in Financial Markets, say a 5-year Credit Default Swap on IBM.

You can purchase this CDS from a Tokyo-based bank, a London-based bank, or a New York-based bank, among other locations. Each of these banks is geographically located in a different part of the world. Depending on the time of year (thanks to daylight savings), Tokyo is 12 hours ahead of New York, and London is 5 hours ahead of New York. Each one of these locations has an official close. For securities that are not housed in a specific location (like a 5-year CDS), there can be "different closes" because the CDSs are not exchange traded but over the counter. As there are OTC markets worldwide, the notion of close is relative to a geographical location.

Reflecting on section 3, think about the quality of your data vendor. Are they providing the time zone, market, currency, etc., of the CDS value? If not, there is uncertainty about where the data originates.

What if we have an exchange, but it never closes? For example, Bitcoin may use GMT - 5 hours as the official close. But for a market that trades 24/7, is this price really more important than the price each day at GMT?

When data lives in different geographical regions, or simply on exchanges with different closing hours, be sure to align properly so that the trading date reflects the close on that date.

6. Complexities in Financial Data

In addition to differing units, timestamps, and currencies, financial data tends to have its own set of complexities. For example, stocks can split, whereby the number of outstanding shares increase and the share price decreases. For example, a company with 100,000,000 shares at \$200 per share could have a 2-for-1 split so that there are now 200,000,000 shares at \$100 per share. Similarly, a company could have a reverse split—those 100,000,000 shares at \$200 could become 50,000,000 shares at \$400 per share. In either case, market capitalization always remains the same. The idea behind splitting is that it makes stocks more affordable. When stocks split, the histories actually have to be adjusted so that the stock doesn't have a 50% drop in price.

In addition, equities receive dividends, which affect the prices, so equity prices also need to be adjusted for the income they receive as dividends. Often, you will see stock prices adjusted for splits and dividends. For example, when importing data from Yahoo Finance, the most suitable column is the "Adjusted Close" column rather than the Close column.

When we upgrade from individual securities to ETFs, we have extra complexities. A sector ETF has members, whose names and weights change over time. So in addition to tracking the price and volume history of ETFs, we may want to track additions, deletions, and re-weightings.

Keep in mind that ignoring deletions creates a survivorship bias. Suppose you did a five-year study on an equity index like S&P500. You start in 2022 and look back 15 years, and you only use those companies that existed for that 15-year period. Companies that started out in 2007 but failed to survive would be excluded from the study, so the study only looks at stocks that survived the 15-year period. This survivorship bias means that our results will appear better than they really were because there are some companies that went bankrupt, bringing the average return down.

Leaving the world of equities, there's a complexity regarding time. Bonds mature. On-the-run active bonds roll. Options expire. Suppose you have a specific 10-year bond issued in the U.S. If you collect this for a number of years, you are following a bond whose time to maturity keeps decreasing. After 10 years, the bond matures.

Yet it is possible to find decades and decades of history of 10-year notes. How? We use a generic 10-year CUSIP.

What makes this generic? Suppose a new 10-year note is issued every three months. Then, every three months, the generic switches from the old issue to the new issue. So long as 10-year notes are regularly issued, we can continue to roll out the old issue into the new one. This is what generic series do. This works for future contracts as well. Since futures expire, we simply use generic versions to track.

Options are the most complex because they don't roll like futures contracts. In the next section, we'll see why options in fact are the most difficult.

7. Data Quality

As financial engineers, we think of data as information provided by markets. Those markets could be securities exchanges, over-the-counter markets, credit agencies, central banks, economic institutions, etc. The data can be provided by the exchanges themselves, broker-dealers with whom we trade, the agencies who make and sell the credit models, central banks, and economic institutions through their API portals. There are also third-party companies that use data to build factor models, or other value-added models that score or rank stocks, securities, factors, and sectors to assist in financial decision-making.

Regardless of the source, there are four features of data quality we always seek:

- Accuracy
- Completeness
- Consistency
- Timeliness

Suppose we were making a lending decision to a consumer. How would we know the credit risks to determine a suitable credit limit, interest rate, and repayment terms? All four of the principles would apply. Here's how the four features would apply (Government Data Quality Hub).

7.1 Accuracy

First, we'd like to ensure that the data we have is about the person to whom we intend to loan funds. We'd want to make sure we have the correct individual and would rely on reference data pointing to the person (a national ID number). Just as securities have their primary keys like CUSIPs or SEDOLs, people should have unique identification. We would want to make sure we don't mix up people with the same name or even financial members with the same name who live at the same address.

7.2 Completeness

Second, we would want a complete history. Imagine if we only pulled credit reports say within the last 10 years for homes and other collateralized loans but not for ordinary credit cards. We would have only a partial picture. Completeness means that the lender has to perform due diligence to discover all information. Of course, regulation may prohibit a lender from getting information that is not protected classes. For example, credit prohibits a person's age or gender as a criterion for getting a loan. These protected classes ensure fairness in lending. Completeness must abide within the confines of what information is allowed.

7.3 Consistency

Third, we would like the data that is provided from different sources to agree. As there are multiple companies that collect and provide credit ratings, we may want to pull from two or more agencies to see if the data is consistent. Indeed, by using multiple credit agencies, we can correct for any discrepancies.

7.4 Timeliness

Fourth, we would like the data to be timely. Suppose a customer applied for six different credit cards all within one morning. We would need to make sure our credit checks reflect real-time lending. Even data that is one day old would not show the extra credit or outstanding balance an individual may have. Such a system could be gamed, and ill-intentioned borrowers can get an inordinate amount of loans all within a short window of time, achieving a balance far beyond what would be accepted if they applied for these loans over a longer period of time. Indeed, the burden on many analytics providers now is that data analytics are performed in real-time. This requires the ability to not only handle real-time requests but also run analyses that take a short time so that decisions can be made quickly. For example, many credit cards offer an opportunity for a credit increase and can provide an answer in under a minute.

The computational and modeling burdens these demands place on analysis require a thorough understanding of the data.

8. The Big Data: The 5 Vs

In the field of data science, machine learning has provided best practices of data for more than 20 years. There is a well-known "Big 5 Vs" (Ishwarappa). We'll discuss these five features:

- Volume**
Volume is the amount of data. Streams and streams of data pour in.
 - Variety**
Variety refers to the different sources and forms. In our previous sections, we saw that data can be structured, semi-structured, or unstructured.
 - Velocity**
Velocity refers to the rate at which data is created in real time. Exchange data can be as fast as fractions of a millisecond! Other financial data may not be available more than once a day or even quarterly in the case of economic indicators.
 - Veracity**
Veracity refers to the confidence in the data source. Think fake news accounts and spoofed trades.
 - Value**
Value refers to the insights available in the data.
- ## 9. Know Your Data
- In finance, when you deal with customers, there's an acronym, KYC: Know Your Customer. In financial engineering, when you deal with data, we can use an analogous acronym, KYD: Know Your Data.
- Here are 10 questions you can ask about the data you collect and intend to use for financial models.
- Does the data have predefined fields?
 - Is the data bounded?
 - Is the data scaled?
 - Is the data observable?
 - Does the data come from a trade, a limit order book, or neither?
 - Is the data quantitative, qualitative, or a hybrid?
 - Is it continuous or categorical?
 - If it is non-numerical, are there language issues?
 - Was the data collected at the frequency with which it was produced?
 - Was the data cleaned or checked by the vendor?
- Let's review these.
- When data has predefined fields, the data structure is easier to identify and maintain. When data lacks predefined fields, a looser data structure is needed.
 - Some data is best left in its raw form, while other data may need to be scaled. Scaling could be as simple as a log transformation or a volume-weighting applied to the price to produce a Volume Weighted Average Price, or VWAP.
 - If data is bounded, then we should acknowledge the lowest possible value and the largest possible value the data could have. If we measured probabilities, then we would expect no value to be negative and no value to be greater than 1. Suppose missing values were coded with -99. Clearly, we would need to recognize this as a missing value and not as data. This is often the mistake with importing data.
 - Many financial data series are directly observable—like stock prices, stock volumes, bond issuance, etc. Ironically, not all data is directly observable. For example, the number of defaults may not be made available, as some data is proprietary. Some data may only be visible based on a given model—for example, default probabilities are not directly observable in the market but are values implied or outputted by credit risk models. The choice of credit risk model will very much affect the default probability we see. The same is true for certain types of volatility—they are not directly observable.
 - Be sure to distinguish trade data—values at which buyers and sellers transacted—from quote data, which are bids, asks, and their sizes. When the bid-ask spread is wide, the midpoint, which is the average of the bid and ask, may not be a good representative of the price.
 - Clearly, qualitative data needs to be processed and interpreted, either through text mining, sentiment analysis, or some other means.
 - Many financial data series are actually categorical. For example, credit spreads are categorical and ordered as well. Others are merely categorical, such as industry classifications. It is easy to confuse the two types when the categories are listed as numbers.
 - Non-standard data is complicated by the fact that it may be in different languages. Sentiment analysis needs to account for parts of language, such as metaphors, figures of speech, expressions, idioms, multiple negations, sarcasm, and even emojis.
 - For exchanges, there is often a wealth of data provided—closing prices, top of book, and various depths of book data bundles. If you are an infrequent trader, say in your own retirement account, you may be inundated by high-frequency data, which provides too much information and would require enormous processing just to make sense at a scale that is more appropriate to the trading decisions you would make. Conversely, daily data would be inappropriate for a market maker. Market makers are subject to intraday volatility because they have to make two-sided markets throughout the day. Volatility at the daily level is too low a frequency to capture what could be a significant event.
 - If you assume data is cleaned from the vendor, then you invite the vendor's gaps, omissions, oversights, misprints, mix-ups, and transpositions into your data, as well as errors in transferring and importing from their side to yours. One way to clean data is to have redundant sources. Imagine you had four sources to tell you a stock's price. Any discrepancies can be identified and handled manually; even better, rules and filters can be applied so that data seems to agree with itself.
- ## 10. Conclusion
- This lesson provided an introduction to the ever-changing role of data in modern finance. We have explored the growing importance of diverse datasets, from traditional market data to alternatives. The key takeaways are:
- Understanding the different types of financial data
 - Identifying various data sources, data identifiers, and descriptors for financial instruments
 - Appreciating complexities around time zones and trading hours
 - Understanding the most significant aspects of data quality
 - Exploring the Vs of big data
- ## References
- Netnod. "What is Precision Time Protocol (PTP)?" <https://www.netnod.se/knowledge-base/What-is-Precision-Time-Protocol-PTP>.
 - Government Data Quality Hub. "Meet the Data Quality Dimensions." *GOV.UK*, 2 July 2020, <https://www.gov.uk/government/news/meet-the-data-quality-dimensions>.
 - Ishwarappa, J. Anuradha. "A Brief Introduction on Big Data 5Vs Characteristics and Hadoop Technology." *Procedia Computer Science*, vol. 48, 2015, pp. 319-324. <https://www.sciencedirect.com/science/article/pii/S1877050915006973?via=ihub>.
- Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you can access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Ozenbas, Deniz, et al. *Liquidity, Markets and Trading in Action*. Springer Link, 2022.
<https://link.springer.com/book/10.1007/978-3-030-74817-3>.

- **Required pages:** Chapter 3, Sections 3.1 to 3.5
- **Estimated reading time:** 30 minutes

financial_data_module_1_lesson_2

December 23, 2025

MODULE 1 | LESSON 2 # GOVERNMENT DATA

Reading Time	2h
Prior Knowledge	matplotlib, python, bonds, interest rates, treasuries
Keywords	government data, Federal Reserve, FRED, API, CMT, Yield Curve, steepening, flattening, slope, level, curvature

In this lesson, we cover government data, which forms the backbone of informed decision-making. We present some key characteristics and aspects of government data while emphasizing the importance of data openness. In Section 2, we focus on the yield curve, more specifically the Constant Maturity Treasury (CMT) yields. We visualize the various phases of a yield curve and examine its dynamics across time. Throughout the lesson, we characterize the yield curve using two fundamental factors: level and slope. This perspective will be invaluable as we progress through the subsequent lessons in this module.

0.1 1. Economics and Financial Data

Asset prices are greatly influenced by what happens in the macroeconomy. A thriving business cycle invites equity investments, increases in personal spending, and perhaps actions taken by the central bank. In the first module of Financial Markets, you saw examples where depositors were earning a specific interest rate. At the same time, lenders (or borrowers) earned a different interest rate. Depositing money in a bank will surely earn less interest than the interest you would pay on a loan from the same bank!

One of the biggest influences on markets in general, and interest rates in particular, is news. News can be thought of as expected or unexpected. Recall that efficient markets may have incorporated much of the expected news already. For example, suppose Netflix declares a \$2.00 dividend. If this is indeed the amount that gets announced, then it comes as no surprise and should not affect Netflix's stock price. However, suppose there is unexpected news.

- Netflix had stronger than anticipated growth due to underestimated popularity with certain shows, causing a larger than expected number of subscribers in Korea. The dividend is \$2.40. The stock price will likely rally.
- On the other hand, suppose Netflix had higher than expected costs for producing original content, causing net profits to drop. The dividend is only \$1.80. The stock price will likely tumble.

Unexpected news can be good or bad; that is, it may result in higher prices or lower prices, depending on the situation. For example, consider bad news. It can increase credit risk, threaten financing cash flows, create volatility, disrupt correlation, aggravate leverage, induce non-linearities, challenge model assumptions, stress models by incurring added costs for liquidity, promote model failure, and even bring about financial crises. Of course, one piece of bad news may not do all that, but what if the bad news affected many businesses, such as higher than expected interest rates?

Let's examine how interest rates work. Investing in business affects interest rates, but simultaneously, interest rates affect business investments. Most relationships in finance are complex because they can be multi-directional. It's not just that A affects B and B affects A. It's that A affects B and C; B affects A and C; C affects A and B; and so on.

For example, if we just collect interest rates and have no sense of monetary policy or money supply or economic decisions, we lose data that both affect interest rates and in turn are affected by those same rates. Indeed, we may want to collect associated data so that we can examine if other variables (e.g., Fed decisions) affect our variable of interest.

0.2 2. The Federal Reserve and Its Impact

Recall from Financial Markets that we read about central banks. The Federal Reserve is the central bank in the United States. The Fed's output is vast: it includes decisions, research, and printed and spoken market commentaries. These findings are heavily watched and analyzed, as they significantly affect markets. For example, eight times a year, the Federal Reserve has regularly scheduled releases of decisions about interest rates. These meetings result in decisions about interest rates released to the market.

Like any other information, this is news that can be expected or unexpected. If the news is expected, then that information is already priced in the market. If the news is unexpected, then there are likely to be information shocks that cause market volatility. In addition, the Fed selects 8 other dates at which they release the minutes of the meetings. These releases contain qualitative rather than quantitative data. These releases can also affect not only interest rates but also stock trading, as they indicate the opinions and intended actions of the Fed. For example, in 2019, the Fed decided to cut interest rates three times. This led to a 29% increase in the S&P 500 during the 4th quarter of 2019.

If we collected data during 2018 and then attempted to extrapolate, we would miss out on key events that affect data. Oftentimes, we need to understand key events that affect our data, as we might consider a data sample as information between those events.

For example, if you construct a sample of equity data, you could collect daily equity closes, but when it comes to analyzing them, you could consider: 1. no grouping 2. 1 group per equity dividend announcement 3. groups based on the level of the Fed Reserve rate 4. 1 group per Fed fund meeting

While the data collected may be the same, there is an acknowledgement that the data as a whole (example 1) may have different properties and can be better analyzed based on an intrinsic property of the company (example 2, earnings) or based on an extrinsic property, like monetary policy (examples 3 and 4). Understanding that our data experiences change means that we will want to understand markets as systems. It also means we may want to collect and analyze data that is qualitative (e.g., Fed statements).

0.3 3. What is Government Data?

Government data is all the information that is collected by government entities. This data include:

- Economic Data (GDP, inflation, employment rates, etc.)
- Demographic Data
- Environmental Data
- Health Data
- Legal Data
- Agriculture Data
- Energy Data
- Education Data
- Real Estate Data

and many more categories of data that a specific government may or may not choose to keep track of. The data above is **essential** for every decision-making mechanism incorporated by the government. For example, it is well established how economic data is important for monetary and fiscal policy or how health data helped the respective agencies in coordinating the healthcare system's response during the COVID-19 pandemic.

In essence, this data serve as evidence-based governance and their holistic view enables governments to make decisions for the future of a country within a complex reality.

0.3.1 3.1 Aspects of Government Data

As of now, we have seen the scope of government data, but there are other characteristics as well:

- Volume: the volume of government data tends to be very large
- Variety: many categories of structured and unstructured data
- Authoritative: it is official (the same data is used for policymaking)
- Timeliness: one can find real-time and historical data
- Quality: there may be incomplete or inaccurate datasets
- Format: data is saved in databases, files (CSV, JSON, etc.), reports, and other formats
- Sensitivity/Accessibility: it may include personal information in which case it will be restricted
- Metadata: the descriptions of the dataset serves as inventory for navigating the large volume of data

Additionally, regarding the above characteristics, there has been a significant effort on behalf of several governments to implement transparency and openness in their data. These efforts gave rise to Open Government Data, which took the form of a movement that pushed many countries to provide public access to their data.

0.3.2 3.2 Open Government Data

In 1966, the U.S. was the first to implement a policy for open data by establishing a legal framework called the “Freedom of Information Act” (FOIA). According to FOIA, any citizen can ask for public access to records from any federal agency. But this was “reactive” transparency since data was not open by default. The latter was only corrected in 2009 by the Open Government Directive, which mandated federal agencies to publish government information online and embrace open government policy. This was the decisive step from a “reactive” transparency to a “proactive” one where anyone could access public data without explicitly requesting to do so. The UK also launched its Open Data Initiative in 2010 followed by the EU Open Data Directive in 2019. Other countries like Canada, Australia, India, and Japan also implemented their respective open data policies.

Here is the definition of Open Government Data as found in the [European Data Portal](#):

Open (Government) Data refers to the information collected, produced or paid for by the public bodies (also referred to as Public Sector Information) and made freely available for re-use for any purpose. The licence will specify the terms of use. These principles for Open Data are described in detail in the [Open Definition](#).

This subset of government data will drive (in part) the datasets that you will use at WQU. These real-world datasets will help you develop practical skills in data analysis and modeling while giving insight into economic indicators and other factors that influence the economy. In summary, transparency in government data has enriched both WQU’s content and the learning process in its entirety.

0.3.3 3.3 Federal Reserve Economic Data (FRED)

A prime example of open government data in action is the Federal Reserve Economic Data (FRED). Currently, FRED stands out as one of the most comprehensive and accessible economic databases available to the public. The data is maintained by the Federal Reserve Bank of St. Louis and given special care, in accordance with the principles of openness, as a crucial resource for researchers, policymakers, students, and the general public.

3.3.1 Overview of FRED FRED is a vast repository of economic time series data from various sources, not limited to the Federal Reserve System. It includes data from:

- U.S. government agencies (e.g., Bureau of Labor Statistics, Bureau of Economic Analysis)
- International organizations (e.g., World Bank, OECD)
- Private sources (e.g., university research centers)

The database covers a wide range of economic indicators, including:

- Gross Domestic Product (GDP)
- Inflation rates
- Employment statistics
- Interest rates
- Exchange rates
- Consumer and producer price indices
- Monetary aggregates

3.3.2 Key Features of FRED

1. Accessibility: FRED data is freely available through a user-friendly web interface, mobile apps, and API access.
2. Data Visualization: Users can create custom charts and graphs directly on the FRED website.
3. Data Download: Data can be easily downloaded in various formats (e.g., Excel, CSV) for further analysis.
4. Regular Updates: FRED is updated in real time as new data becomes available from its sources.
5. Historical Data: Many series in FRED go back several decades, providing valuable historical context.
6. Data Transformation: FRED allows users to perform basic transformations on data series, such as changing units or calculating percent changes.

For the purposes of this lesson, we use the API gateway in order to make use of the statistical and visualization libraries of Python.

0.4 4. Yield Curve

In this section, we will work with a specific type of yield curve called the Constant Maturity Treasury (CMT) yield curve. In Financial Markets, students already got an introduction to this subject, and we will be supplementing that material with a hands-on data workflow.

The CMT yield curve is a snapshot of all U.S. Treasury yields at a given date. Essentially, the U.S. Treasury uses the closing market bid yields (which are based on the prices that buyers are willing to pay (bid) instead of the actual transaction prices) in order to calculate CMT rates on a daily basis. You can find the overview of the methodology that the Treasury uses [here](#).

Conveniently, FRED has already performed these calculations and the CMT rates are freely available from their website or through the FRED API, which we can utilize through the `fredapi` Python library. In order to access the FRED data, every student must obtain an API key by following these steps:

1. Visit the FRED website: https://fred.stlouisfed.org/docs/api/api_key.html. Click on “Create an Account” in the top right corner (if you do not already have an account with FRED).
2. Log in to your FRED account and request an API key. Navigate to the “API Keys” selection of the “My Account” dropdown menu in the top right corner. Then, click on “Request API Key.”

```
[1]: import pandas as pd
      from fredapi import Fred
      import matplotlib.pyplot as plt
      import seaborn as sns
      sns.set()
```

```
[2]: # Initialize the FRED API with your key
      fred = Fred(api_key='99b15e0a2f3b3f4571893e831fd555d0') # Replace my APIKEY_
      ↪with "YOUR_API_KEY"

      # List of Treasury yield series IDs
```

```

series_ids = ['DGS1M0', 'DGS3M0', 'DGS6M0', 'DGS1', 'DGS2', 'DGS3', 'DGS5', \
              'DGS7', 'DGS10', 'DGS20', 'DGS30']

# Function to get data for a single series
def get_yield_data(series_id):
    data = fred.get_series(series_id, observation_start="1975-01-01", \
        ↪observation_end="2024-05-03")
    return data

# Get data for all series
yields_dict = {series_id: get_yield_data(series_id) for series_id in series_ids}

# Combine into a single DataFrame
yields = pd.DataFrame(yields_dict)

# Rename columns for clarity
yields.columns = ['1 Month', '3 Month', '6 Month', '1 Year', '2 Year', '3 \
    ↪Year', '5 Year', \
                  '7 Year', '10 Year', '20 Year', '30 Year']

```

```

[3]: yields.index = pd.to_datetime(yields.index)
yields.loc['2020-01-03']

```

```

[3]: 1 Month      1.52
     3 Month      1.52
     6 Month      1.55
     1 Year       1.55
     2 Year       1.53
     3 Year       1.54
     5 Year       1.59
     7 Year       1.71
    10 Year       1.80
    20 Year       2.11
    30 Year       2.26
     Name: 2020-01-03 00:00:00, dtype: float64

```

The above dataframe contains one row of our dataset: all the CMT yields at a specific date.

The yields are expressed as percentages. For example, the “1 Month” yield is at 1.52, which means 1.52% or 0.0152, and the precision with which the yields are presented is one basis point (0.01%).

```

[4]: yields.isna().sum(axis = 0)

```

```

[4]: 1 Month      7180
     3 Month      2204
     6 Month      2204
     1 Year        542
     2 Year         894

```

```

3 Year      542
5 Year      542
7 Year      542
10 Year     542
20 Year    2231
30 Year    1072
dtype: int64

```

The first thing we notice is the missing data. Some maturities such as the 1 month, 3 month, and 6 month were not tracked over time because before the 2000s, the focus was more on longer-term maturities. The 20-year bond had an irregular issuance history because from 1986 to 1993, 20-year T-bonds were not issued. The same applies to the 30-year maturity, which was discontinued from February 2002 to February 2006. Furthermore, a great deal of missing values, as one can see with the more liquid bonds like the 3-, 5-, 7-, and 10-year maturities, can be attributed to holidays and market disruptions.

```

[5]: # We can see that during the days that the 10-year is not reported,
      # none maturity is reported as well.
      yields[yields['10 Year'].isna() == True].sum(axis = 0)

```

```

[5]: 1 Month      0.0
      3 Month      0.0
      6 Month      0.0
      1 Year       0.0
      2 Year       0.0
      3 Year       0.0
      5 Year       0.0
      7 Year       0.0
      10 Year      0.0
      20 Year      0.0
      30 Year      0.0
      dtype: float64

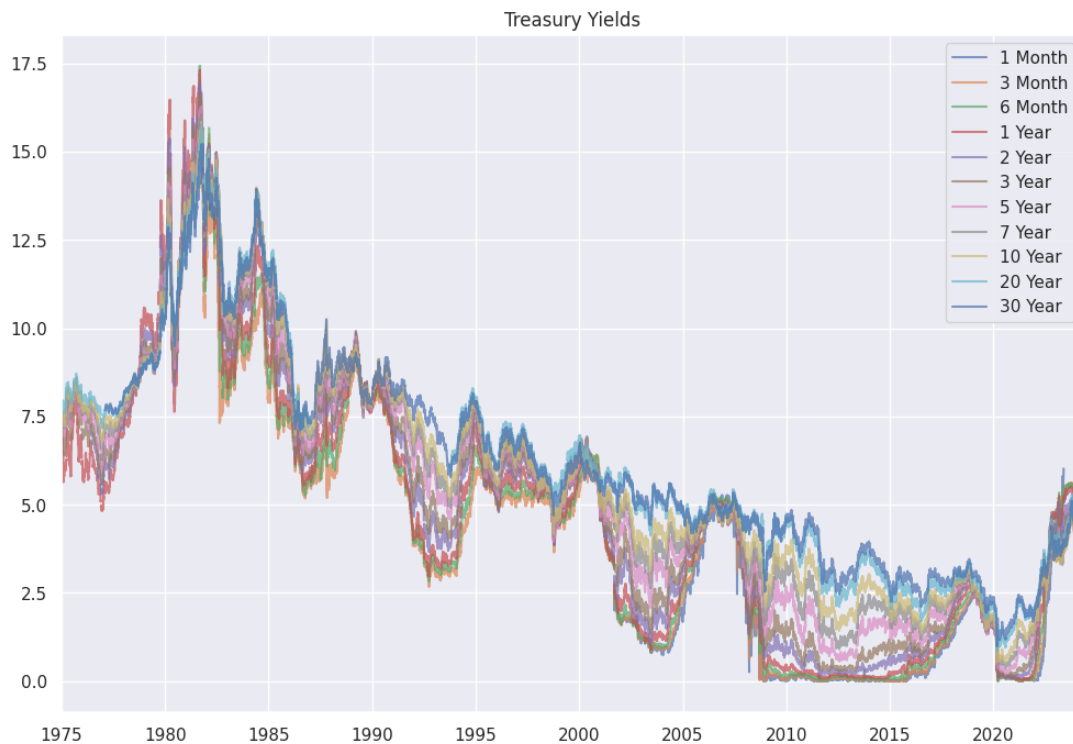
```

Let's plot the yields for all the available dates. Even though our dataset becomes "complete" only after the 2000s, it is important to observe how rates evolved over time.

```

[6]: # Figure 1
      yields.plot(figsize=(12, 8), title='Treasury Yields', alpha=0.7) # Plot the
      ↪ yields
      plt.show()

```



Exercise 1

List all the U.S. economic recessions post 1975. What do you observe about the rates during these recessions? Use the graph above to argue for how rates behave during economic downturns. In order to make things more clear, you can select one or two of the U.S. recessions and recreate the above graph just for the duration of the crisis (and a few years prior to and after the year of the crisis).

Let's also visualize the interdependence between yields:

```
[7]: # Figure 2
sns.pairplot(yields.dropna(), diag_kind='kde') # Pairplot with KDE on the
↪diagonal
plt.suptitle("Pairwise Distribution and KDE of Daily Yield Data (2001-07-31 to
↪2024-05-03)",
            y=1.02, fontsize=26)
plt.show()
```

Pairwise Distribution and KDE of Daily Yield Data (2001-07-31 to 2024-05-03)



The pairplot above displays the relationship between the U.S. Treasuries of different maturities. On the diagonal, we can see the kernel density estimation (KDE) of each yield while the off-diagonal plots show the scatterplots for each pair.

The first thing we notice is that the KDEs of the yields are multimodal. This reflects the effect of various economic regimes, cycles, changes in policy, and significant events on the yields.

Additionally, the scatterplots show strong positive correlations between the yields. More specifically, the yields whose maturities are close have perfect correlation (close to 1) whereas the far-away yields are still positively correlated even though not perfectly.

Let's visualize the yields at a specific date:

[8]: *# Figure 3*

```
def plot_yield_curve(date):
    maturities = ['1M', '3M', '6M', '1Y', '2Y', '3Y', '5Y', '7Y', '10Y', '20Y',
↪ '30Y'] # Maturities
    fig, ax = plt.subplots(figsize=(12, 6))
    ax.plot(maturities, yields.loc[date], marker='D', label='Yield Curve at ' +
↪ date)

    ax.set_yticklabels(['{: .2f}%'.format(y) for y in ax.get_yticks()])
    ax.set_xticks(range(len(maturities)))
    ax.set_xticklabels(maturities)

    # Add labels and title
    ax.set_xlabel('Maturity')
    ax.set_ylabel('Yield')
    ax.set_title('Treasury Yield Curve')

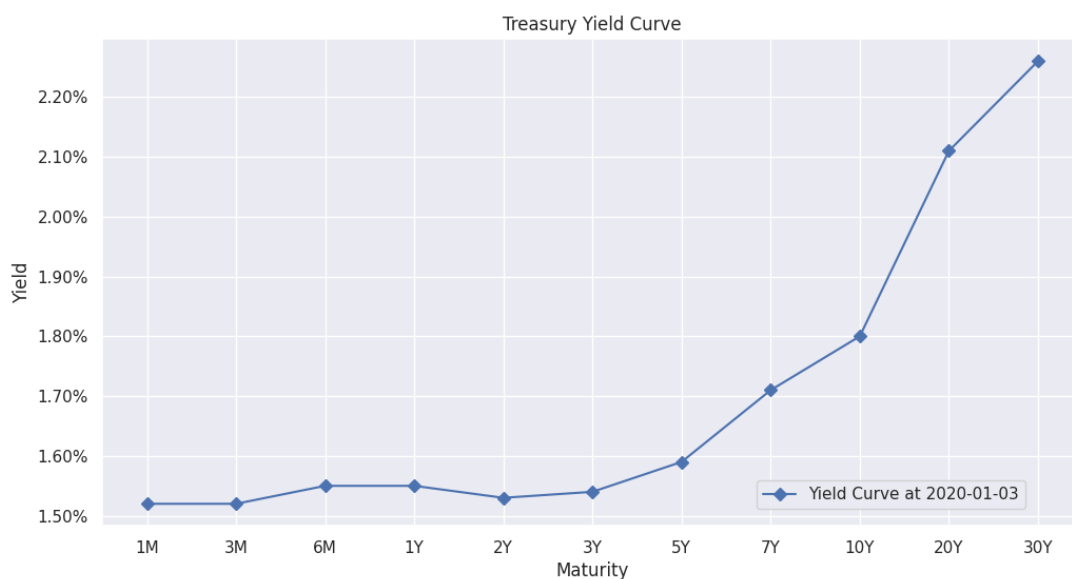
    fig.legend(loc = [0.69, 0.14])

    # Show the plot
    plt.grid(True)
    plt.show()

plot_yield_curve('2020-01-03')
```

/tmp/ipykernel_27/2521112310.py:8: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.

```
ax.set_yticklabels(['{: .2f}%'.format(y) for y in ax.get_yticks()])
```



Exercise 2

The graph above shows all the yields at 2020-01-03. Find (one or two) specific dates that will help you recreate the above plot twice: once for a flattened curve and once for an inverted curve. Argue how the flat and inverted curves correspond to specific phases of Figure 1 (in several cases, the combined yield line plots look thin). Why do those dates matter?

0.4.1 4.1 The Shape of the Yield Curve

By examining Figure 3 along with the figures that you will have produced by now in answering Exercise 2, you have observed that the (most obvious) factors that define the shape of a yield curve are (Rebonato): 1. The overall **level** of the yield curve, which translates into how high or how low interest rates are. 2. The **slope** of the yield curve, which indicates how steep, flat, or inverted the curve is.

Typically, we can explain about 90% of the variability of the curve through level and slope (Oprea). For a more thorough analysis, one would need to take into account: 3. The **curvature**, which is a measure of how the middle of the curve (medium-term rates) behave in comparison to the short- and long-term rates.

For the purposes of this lesson, we will emphasize the first two factors.

4.1.1 Level of the Curve The level is associated with **parallel shifts** in the entire yield curve. In essence, it affects all bond prices regardless of maturity by (almost) the same magnitude. These *shifts* are driven by macroeconomic factors such as inflation, geopolitical events, changes in monetary policy, etc.

We will examine the 1994 Bond Market Crash, also known as the Great Bond Massacre. The primary trigger of the bond crash was the Fed's decision to initiate a round of rate hikes. In February 1994, the Fed raised the funds rate from 3.00% to 3.25%. Over the course of the year, the Fed continued to raise rates incrementally, reaching 5.5% by February of 1995. The main driver behind the Fed's decision was inflation expectations due to the rapid economic expansion of 1993.

Additionally, prior to 1994, the spread between the short-term and long-term rates was more than 4 percentage points, leading many investors in leveraged positions to borrow short term and invest in longer-term bonds. Naturally, the sudden rate hikes caught these investors (among others) off guard, forcing them to unwind their position, leading to a selloff in the bond market.

```
[9]: # Figure 4

from matplotlib.gridspec import GridSpec

# Preparing the data
bond_market_crash = yields.loc['1994-01-03': '1994-11-30'].iloc[:,1:] * 100 #_
    ↪ Convert yields to percentage
resampled_monthly_bond_market_crash = bond_market_crash.resample('M').last()
resampled_monthly_bond_market_crash_ticks = resampled_monthly_bond_market_crash.
    ↪ index.strftime('%Y-%m-%d')
```

```

# Creating the plt object
fig = plt.figure(figsize=(16, 12))
gs = GridSpec(2, 2, height_ratios=[1, 1.5])

# First row, first column
ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(bond_market_crash, alpha=0.7)
ax1.set_title('Treasury Yields across time - Graph 1')
ax1.set_xlabel('Date')
ax1.set_ylabel('Yield (%)')
ax1.tick_params(axis='x', rotation=45)

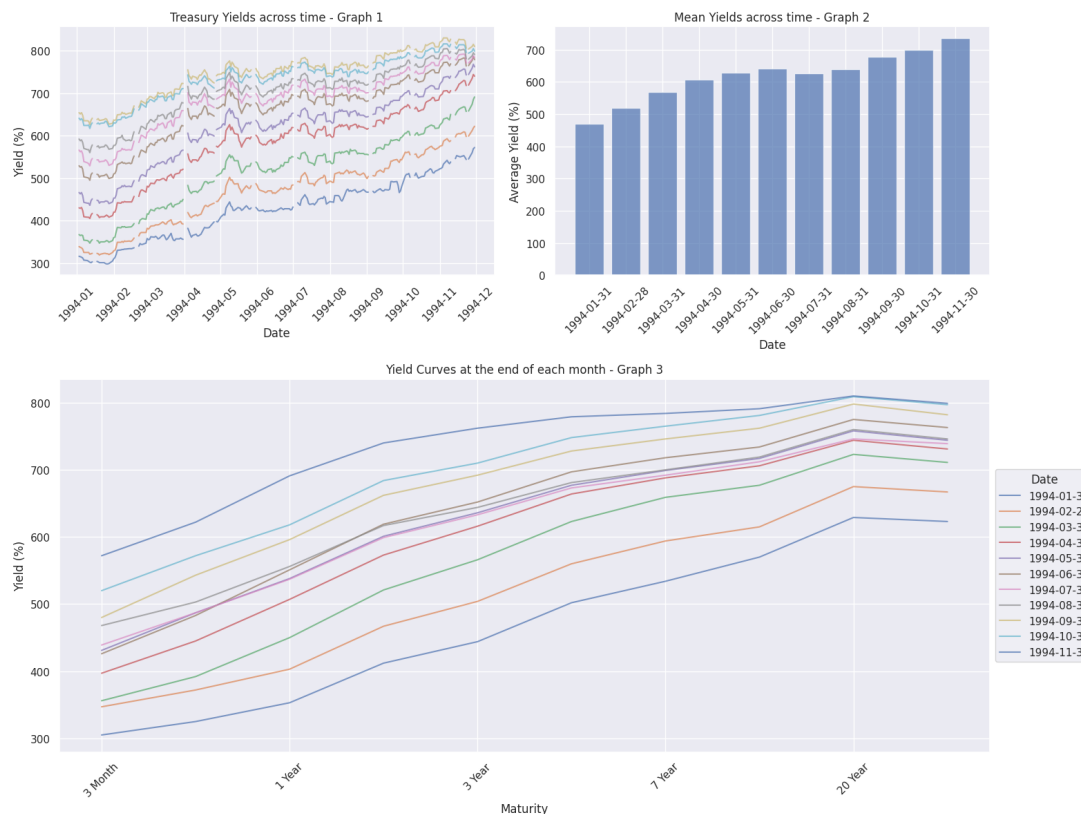
# First row, second column
ax2 = fig.add_subplot(gs[0, 1])
ax2.bar(resampled_monthly_bond_market_crash_ticks,
        ↪resampled_monthly_bond_market_crash.mean(axis=1), alpha=0.7)
ax2.set_title('Mean Yields across time - Graph 2')
ax2.set_xlabel('Date')
ax2.set_ylabel('Average Yield (%)')
ax2.tick_params(axis='x', rotation=45)

# Second row
ax3 = fig.add_subplot(gs[1, :])
for i in range(len(resampled_monthly_bond_market_crash)):
    date = resampled_monthly_bond_market_crash_ticks[i]
    resampled_monthly_bond_market_crash.iloc[i, :].plot(ax=ax3, alpha=0.7,
    ↪label=date)

ax3.set_title('Yield Curves at the end of each month - Graph 3')
ax3.set_xlabel('Maturity')
ax3.set_ylabel('Yield (%)')
ax3.legend(title='Date', loc='center left', bbox_to_anchor=(1, 0.5))
ax3.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

```



In Figure 4, one can see the consistent upward movement of the yields across all maturities. Specifically in Graph 3, we observe every yield curve at the end of each month during 1994 (the last day of each month was arbitrarily selected as a representative of the month). This is a prime example of a change in the level of the yields, where the entire curve is moving upwards each month while keeping its slope intact.

4.1.2 Slope of the Yield Curve The slope of the yield curve typically refers to the difference in yields between the long-term and the short-term bonds. We often use the difference between the 10-year and the 2-year or between the 30-year and the 3-month bonds, but depending on the context, one can choose different maturities. The slope can be categorized into: 1. A positive slope, also referred to as normal yield curve, which occurs when long-term yields are higher than the short-term ones. This is the “normal” scenario since we expect that the additional risk of holding long-term bonds should be compensated with higher returns than those of the short-term ones. 2. A flat slope: short-term and long-term yields are similar. 3. A negative slope, also referred to as inverted yield curve. It occurs when short-term yields are higher than long-term ones.

As we have seen up until now, the slope can be easily seen in a yield curve graph. But what about the dynamics of slope changes?

Let's assume we are in a position where the yield curve is normal. From this point, the slope can either steepen or flatten, and in each case, this can be done in multiple ways: 1. Steepening * Bull steepening: short-term rates fall faster than long-term rates * Bear steepening: long-term rates

increase faster than short-term rates 2. Flattening * Bull flattening: long-term rates fall faster than short-term rates * Bear flattening: short-term rates increase faster than long-term rates

In Figure 5, you can see a visual representation of the flattening dynamics of the curve.

```
[10]: # Figure 5

# Preparing the data
bear_flattening = yields['2004-06-01':'2006-06-30'].resample('3M').last()
bear_flattening.index = bear_flattening.index.strftime('%Y-%m-%d')

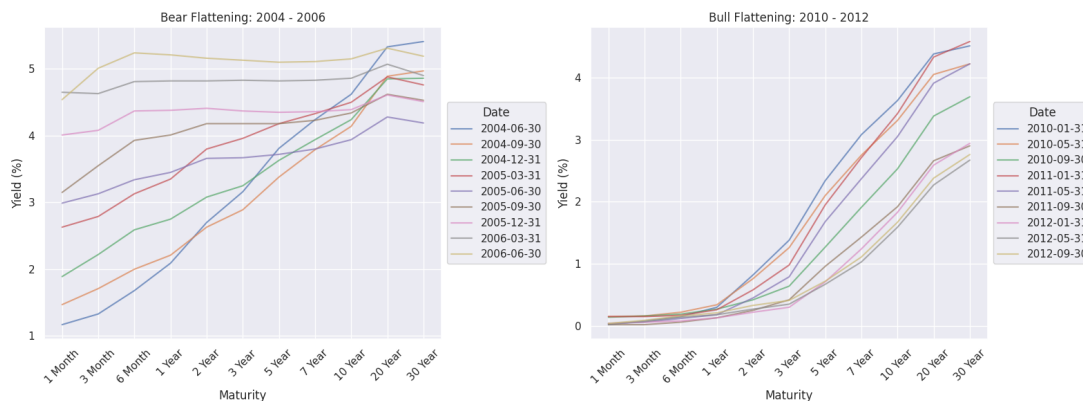
bull_flattening = yields['2010-01-01':'2012-06-30'].resample('4M').last()
bull_flattening.index = bull_flattening.index.strftime('%Y-%m-%d')

# Creating the plt object
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Bear flattening graph
for i in range(len(bear_flattening)):
    date = bear_flattening.index[i]
    ax1.plot(bear_flattening.columns, bear_flattening.iloc[i, :], alpha=0.7,
             label=date)
ax1.set_title('Bear Flattening: 2004 - 2006')
ax1.set_xlabel('Maturity')
ax1.set_ylabel('Yield (%)')
ax1.legend(title='Date', loc='center left', bbox_to_anchor=(1, 0.5))
ax1.tick_params(axis='x', rotation=45)

# Bull flattening graph
for i in range(len(bull_flattening)):
    date = bull_flattening.index[i]
    ax2.plot(bull_flattening.columns, bull_flattening.iloc[i, :], alpha=0.7,
             label=date)
ax2.set_title('Bull Flattening: 2010 - 2012')
ax2.set_xlabel('Maturity')
ax2.set_ylabel('Yield (%)')
ax2.legend(title='Date', loc='center left', bbox_to_anchor=(1, 0.5))
ax2.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```



Exercise 3

Recreate Figure 5 to illustrate the bull and bear steepening. Find the periods that correspond to the steepening of the yield curve by searching online or in academic textbooks, and utilize the code provided to demonstrate the dynamics of the steepening. Make an argument about the economic conditions and the response of the Fed during these times. Paste the plot in the forum along with a small paragraph that explains what we see.

Exercise 4

Demonstrate the importance of the difference between 10-year and 2-year bonds. Create a double y-axis plot where you superimpose the S&P500 on the difference between the two bonds.

0.5 5. Conclusion

Through hands-on analysis and visualization using Python and the FRED API, we examined the dynamics of the yield curve over time. We explored how factors such as the level and slope shape the yield curve and how these elements reflect and influence broader economic conditions.

Key takeaways include:

- The importance of open government data for research, policymaking, and public understanding of economic trends
- The significance of the yield curve as an economic indicator, including its various shapes (normal, flat, and inverted) and what they may signify about economic expectations
- The concepts of yield curve steepening and flattening and how these movements can be categorized as “bull” or “bear” depending on the underlying rate changes

References

- Rebonato, Riccardo. *Bond Pricing and Yield Curve Modeling: A Structural Approach*. Cambridge University Press, 2018.
- Oprea, Andreea, *The Use of Principal Component Analysis (PCA) in Building Yield Curve Scenarios and Identifying Relative-Value Trading Opportunities on the Romanian Government Bond Market*, Journal of Risk and Financial Management, 2022, 15(6), 247

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. LibreTexts Mathematics. "5.05 Spline Method of Interpolation."

[https://math.libretexts.org/Workbench/Numerical_Methods_with_Applications_\(Kaw\)/5%3A_Interpolation/5.05%3A_Spline_Method_of_Interpolation](https://math.libretexts.org/Workbench/Numerical_Methods_with_Applications_(Kaw)/5%3A_Interpolation/5.05%3A_Spline_Method_of_Interpolation).

- **Required pages:** Lessons 1–5
- **Estimated reading time:** 60 minutes

GOVERNMENT BOND YIELD CURVE ANALYSIS

Reading Time 60 minutes

Prior Knowledge U.S. Treasury Bonds, Yield Curve, Linear Algebra, Basic Python

Keywords Bond price-yield curve, Risk free interest rate, Nelson Siegel, Cubic Spline

In the previous lesson, we learned how to obtain U.S. Treasury yield information and went through some foundational concepts about yield curve. In this lesson, we are going to dive deeper and learn how to fit a bond yield curve. We will continue to use U.S. Treasury yield data for demonstration.

1. Risk-Free Interest Rates

In the Financial Markets course, we learned about credit risk with investing in bonds. Credit risk, which is assumed by the bond investor, is the risk that the bond issuer will default on the bond. An entity with a high credit rating or credit score can borrow money from financial markets at a lower interest rate compared to an entity with a lower credit rating. In the U.S., government-issued bonds (U.S. Treasuries or Treasuries) are usually considered high credit rating bonds. Therefore, Treasury yields are usually used as **risk-free rates** for bond price evaluations in the financial industry. There are a lot of debates about whether U.S. Treasuries are actually safe with no risk of default. Different scenarios will need different considerations. In this lesson, we'll follow current financial market convention by using U.S. Treasury yields as risk-free rates.

Another key concept in credit risk is credit spread. **Credit spread** or **credit premium** is the interest rate difference between a corporate bond and a government bond of the same maturity. This interest rate difference is also called **excess return** of the corporate bond since the government bond yield is risk-free interest. The excess return of the corporate bond is the additional return to a corporate bond holder to take on additional risk in holding this corporate bond.

Since risk-free interest rates are a key element not only in bond pricing but also in other financial asset pricing and portfolio management, it is crucial to understand the characteristics and behavior of risk-free interest. In this lesson, we will use U.S. treasury (or

treasuries) yields as risk-free interest to investigate their behavior and conduct analysis.

2. Volatility of U.S. Treasury Yields

First, let's take a look of the volatilities of Treasury yields of different maturities. We continue from Lesson 2's method to get Treasury yield data from FRED using Python as follows.

```
In [1]: import pandas as pd
        from fredapi import Fred
        import matplotlib.pyplot as plt
        import seaborn as sns
        sns.set()
```

```
In [2]: # Initialize the FRED API with your key
        fred = Fred(api_key='5079f41d061a4037d81f3da69e018803') # Replace my APIKEY with "Y

        # List of Treasury yield series IDs
        series_ids = ['DGS1M0', 'DGS3M0', 'DGS6M0', 'DGS1', 'DGS2', 'DGS3', 'DGS5', \
                      'DGS7', 'DGS10', 'DGS20', 'DGS30']

        # Function to get data for a single series
        def get_yield_data(series_id):
            data = fred.get_series(series_id, observation_start="1975-01-01", observation_e
            return data

        # Get data for all series
        yields_dict = {series_id: get_yield_data(series_id) for series_id in series_ids}

        # Combine into a single DataFrame
        yields = pd.DataFrame(yields_dict)

        # Rename columns for clarity
        yields.columns = ['1 Month', '3 Month', '6 Month', '1 Year', '2 Year', '3 Year', '5
                          '7 Year', '10 Year', '20 Year', '30 Year']
```

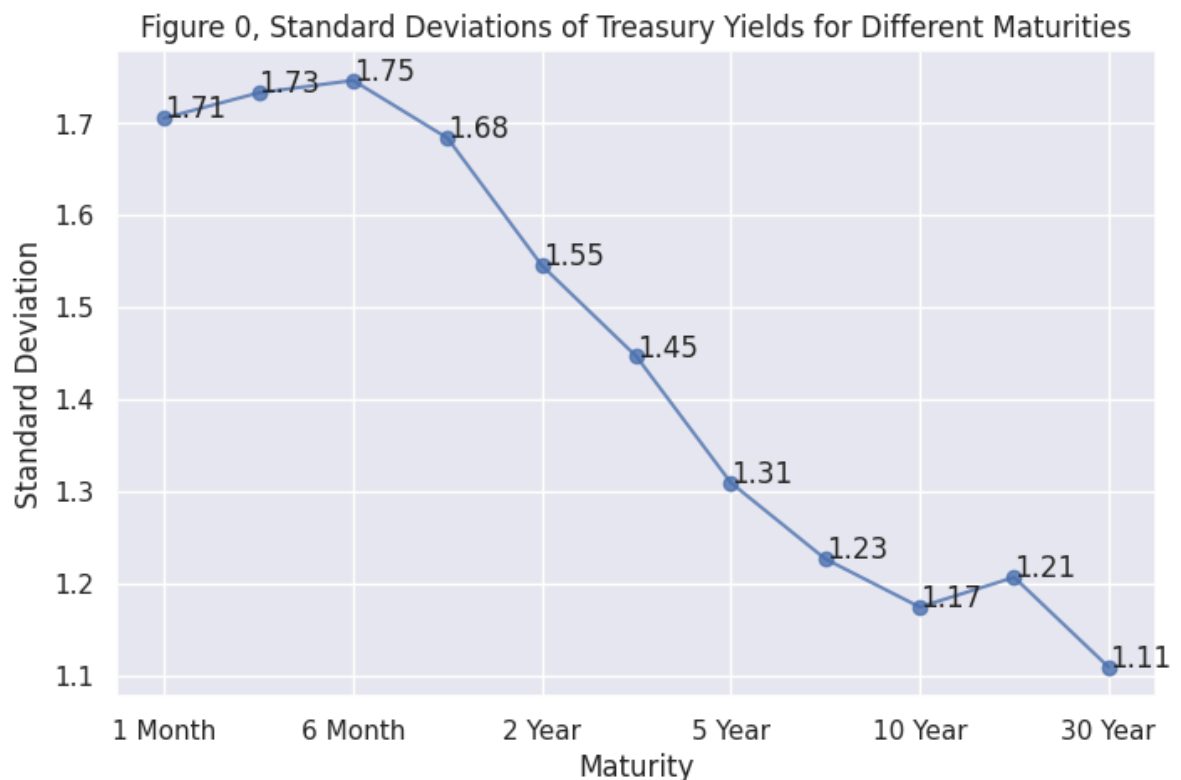
```
In [3]: yields.index = pd.to_datetime(yields.index)
```

Now let's calculate the standard deviations of Treasury yields of different maturities. Then, we'll draw a graph to present the standard deviations by maturity.

```
In [4]: yields = yields.dropna()
        y_std = yields.std()
        y_std
```

```
Out[4]: 1 Month      1.705851
        3 Month      1.733347
        6 Month      1.746829
        1 Year       1.684091
        2 Year       1.545413
        3 Year       1.446639
        5 Year       1.309345
        7 Year       1.226457
        10 Year      1.174092
        20 Year      1.206514
        30 Year      1.108774
        dtype: float64
```

```
In [5]: fig, ax = plt.subplots()
y_std.plot(figsize = (8,5),marker='o', title='Figure 0, Standard Deviations of Trea
plt.xlabel("Maturity")
plt.ylabel("Standard Deviation")
for i in range(len(y_std)):
    ax.annotate(str(round(y_std.iloc[i],2)),xy=(i,y_std.iloc[i]))
plt.show()
```



From the above graph, we see that the standard deviations of U.S. Treasury bonds with short maturities are higher than the U.S. Treasury bonds with longer maturities. The standard deviations for Treasury bonds with maturities of less than one year stay high above 1.7. Then, the standard deviations of Treasury bonds with maturities over one year decrease steadily.

3. Treasury Bond Price-Yield Curve

According to the bond price mathematical formula, the relationship between bond price and bond yield is non-linear. This relationship is best explained with a bond price-yield curve. Let's use the following figure to demonstrate the relationship.

Figure 1: Bond Price-Yield Curve

 Graph showing price vs. yield with a line intersecting a curve at a marked point

In Figure 1, we can see that when the yield/interest of a bond increases, the price of the bond will decrease. Conversely, when the yield/interest of a bond decreases, the price of the bond will increase. There is a negative relationship between the bond price and bond yields. However, when yield changes by one unit, the price change varies depending on where the yield level is when yield change happens.

This is another key point to pay attention to in the bond price-yield relationship: the bond price and bond yield do not have a linear relationship; they have a convex relationship. This non-linear relationship between bond price and bond yield has an important implication in interest rate risk management for bond investment. Let's use the following Figure 2 to explain the concept.

Figure 2: Non-Linear Relationship between Bond Price and Bond Yield

 Graph showing price vs. yield with two rectangles (D1, D2) illustrating changes in price (P1, P2) along a downward-sloping curve

Based on the convex nature of the bond price-yield curve depicted in Figure 2, the amount of price change when yield changes by the same amount varies depending on yield level. In Figure 2, D1 and D2 have the same length. When the yield changes by D1, the amount of price change is P1. When the yield changes by D2, the amount of price change is P2. However, the yield level for D1 is higher than the yield level for D2. Because of the convexity of the price yield curve, P1 is smaller than P2.

The feature of this relationship between bond price and bond yield is called **curvature**. The example above demonstrates that the slope of the price-yield curve changes when the level of the yield changes. The curvature of the price-yield curve is used to describe this slope dependency on yield level. A bond portfolio manager will have to pay attention to the curvatures of the bond holdings when managing interest rate risk.

4. Polynomial Fitting for U.S. Treasury Yield Curve

We talked about U.S. Treasury yield curve in the previous lesson. Yield curve describes the relationship of bond yield and time to maturity (or maturity) of similar bonds. This relationship is also called the **term structure** of a yield curve. Analyzing term structure is critical in bond pricing and interest rate risk management. In this section, we will introduce

methods to fit a yield curve. We will continue to use U.S. Treasury bond yields as an example. First, let's make a graph for the U.S. Treasury yield curve on 2020 January 10th.

```
In [6]: def plot_yield_curve(date, fig_n):
    maturities = ['1M', '3M', '6M', '1Y', '2Y', '3Y', '5Y', '7Y', '10Y', '20Y', '30Y']
    fig, ax = plt.subplots(figsize=(12, 6))
    ax.plot(maturities, yields.loc[date], marker='D', label='Yield Curve at ' + date)

    ax.set_yticklabels(['{:0.2f}%'.format(y) for y in ax.get_yticks()])
    ax.set_xticks(range(len(maturities)))
    ax.set_xticklabels(maturities)

    # Add labels and title
    ax.set_xlabel('Maturity')
    ax.set_ylabel('Yield')
    ax.set_title(fig_n+'Treasury Yield Curve')

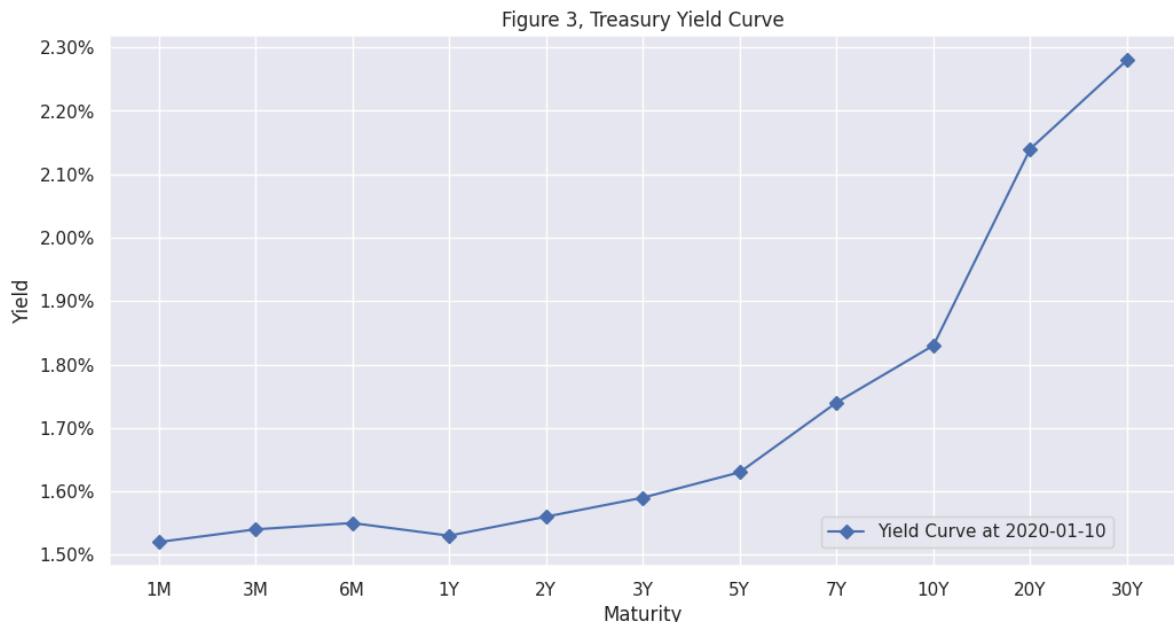
    fig.legend(loc = [0.69, 0.14])

    # Show the plot
    plt.grid(True)
    plt.show()

print("Figure 3")
plot_yield_curve('2020-01-10', 'Figure 3, ')
```

Figure 3

```
/tmp/ipykernel_29/1226209266.py:6: UserWarning: set_ticklabels() should only be used
with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_yticklabels(['{:0.2f}%'.format(y) for y in ax.get_yticks()])
```



In this section, we are going to use a polynomial fitting technique to estimate a yield curve.

Polynomial fitting is when a researcher uses the n th degree of input variable polynomials to predict the output variable. For example, if the input variable is x and the output

variable is y , the following expression is to use the n th degree of x polynomials to predict y .

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x^2 + \beta_3 x^3 \dots + \beta_n x^n$$

$\beta_0, \beta_1, \beta_2, \beta_3, \dots, \beta_n$ are parameters to be estimated.

There are several polynomial fitting methods to fit a yield curve. In this lesson, we are going to learn two methods: the **Nelson Siegel model** and **cubic spline fitting**.

4.1 Nelson Siegel Model

The **Nelson Siegel model (NS model)** is a popular model for describing the relationship between maturity and yield (Svensson). Here is the formula for the model:

$$y(t) = \beta_0 + \beta_1 \left(\frac{1 - e^{-\lambda t}}{\lambda t} \right) + \beta_2 \left(\frac{1 - e^{-\lambda t}}{\lambda t} \right)^2 + \epsilon$$

$\beta_0, \beta_1, \beta_2$ are the parameters to be estimated. t is the time to maturity and λ is the decay rate. The decay rate is between 0 and 1. β_0 is used to describe the level of the yield curve. β_1 is used to describe the slope of the yield curve and β_2 is used to describe the shape of the yield curve. For this reason, we also call the NS model a **yield curve factor model**. The NS model decomposes the yield curve into three elements as described above.

How does the decay rate work? The smaller the decay rate, the slower the curve decays. The larger the decay rate, the faster the curve decays. The decay rate shows how fast the yield will converge to the long-term average.

With the NS model's simple structure, we can use different elements in the model to describe different yield curve behaviors. Once we have an estimated NS model, we can use the model to predict the action of future interest rate moves (Pape).

Let's use the Nelson Siegel Svensson package from Python to demonstrate the NS model.

4.2 Nelson Siegel Model: Python Demonstration

In this section, we are going to use the Nelson Siegel Svensson package from Python to show how to fit a yield curve. First, we need to install and import the package.

```
In [7]: pip install nelson_siegel_svensson
```

Requirement already satisfied: nelson_siegel_svensson in /usr/local/lib/python3.11/site-packages (0.5.0)
 Requirement already satisfied: Click>=8.0 in /usr/local/lib/python3.11/site-packages (from nelson_siegel_svensson) (8.1.7)
 Requirement already satisfied: numpy>=1.22 in /usr/local/lib/python3.11/site-packages (from nelson_siegel_svensson) (1.26.2)
 Requirement already satisfied: scipy>=1.7 in /usr/local/lib/python3.11/site-packages (from nelson_siegel_svensson) (1.11.4)
 Requirement already satisfied: matplotlib>=3.5 in /usr/local/lib/python3.11/site-packages (from nelson_siegel_svensson) (3.8.2)
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->nelson_siegel_svensson) (1.2.0)
 Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->nelson_siegel_svensson) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->nelson_siegel_svensson) (4.46.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->nelson_siegel_svensson) (1.4.5)
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->nelson_siegel_svensson) (23.1)
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->nelson_siegel_svensson) (9.3.0)
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->nelson_siegel_svensson) (3.1.1)
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/site-packages (from matplotlib>=3.5->nelson_siegel_svensson) (2.8.2)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages (from python-dateutil>=2.7->matplotlib>=3.5->nelson_siegel_svensson) (1.16.0)
 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip available: 22.3.1 -> 25.2

[notice] To update, run: `pip install --upgrade pip`

Note: you may need to restart the kernel to use updated packages.

```
In [8]: # Import the packages for fitting NS model
from nelson_siegel_svensson.calibrate import calibrate_ns_ols
import numpy as np
```

After importing the necessary packages for fitting the NS model, we need to create variables for modeling. We first need to create a maturity variable t in terms of year. For example, 1 month is 0.08333 year and 3-month is 0.25 year. The next variable is the yield variable. Both maturity variable and yield variable need to be in an array form. We are going to use the yields from 2020 January 10th (2020-01-10) as our example since we just drew the yield curve on that day in the last section.

```
In [9]: # Create maturity and yield variables in array form
t = np.array([0.08333, 0.25, 0.5, 1, 2, 3, 5, 7, 10, 20, 30])
y = np.array(yields.loc["2020-01-10"])
```

Once the variables are ready, we can move to estimate the NS model.

```
In [10]: # Fit an NS model for yields from 2020-01-10
curve, status = calibrate_ns_ols(t, y, tau0=1.0) # starting value of 1.0 for the o
assert status.success
print(curve)
```

```
NelsonSiegelCurve(beta0=2.6376812673219825, beta1=-1.1025863779183895, beta2=-1.1406
470007593532, tau=4.750020999234434)
```

The NS model result shows the estimated β_0 , β_1 , β_2 , and decay rate. Now let's plot the graph to better understand the model result.

```
In [11]: y_hat = curve
t_hat = np.linspace(0.5, 30, 100)
plt.plot(t_hat, y_hat(t_hat))
plt.xlabel("Maturity")
plt.ylabel("Yield")
plt.title("Figure 4, NS Model Result")
```

```
Out[11]: Text(0.5, 1.0, 'Figure 4, NS Model Result')
```

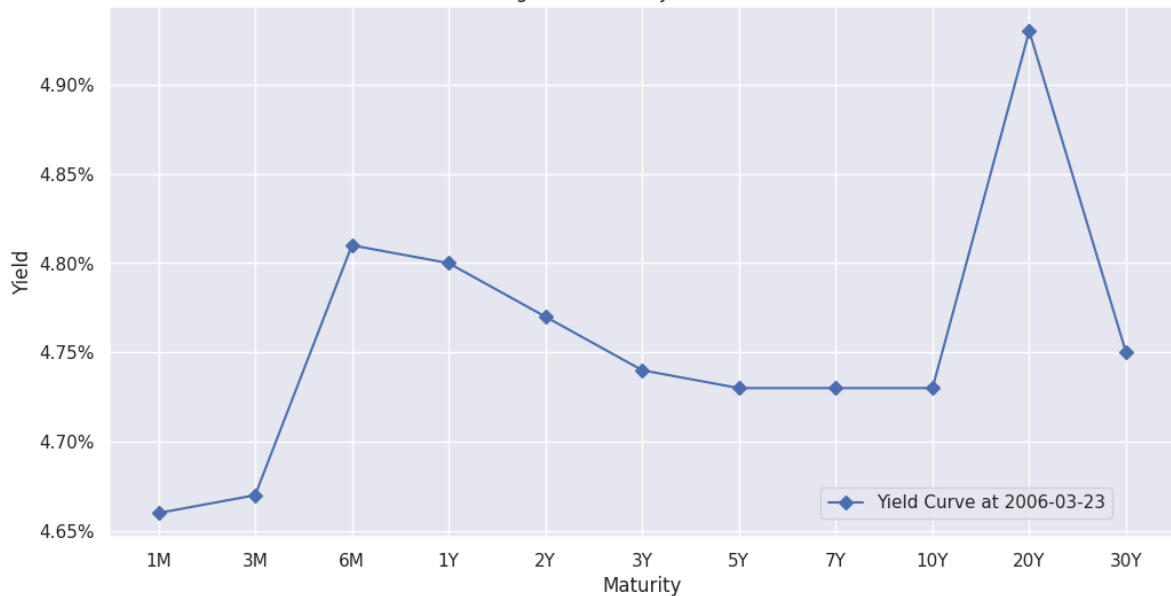


From the above Figure 4, we can see the estimated yield curve is a pretty good fit of the curve chart we drew in the previous section. Let's estimate a different shape of yield curve. This time, we'll use the yields from 2006 March 23rd (2006-03-23). First, let's draw the yield data from 2006-03-23.

```
In [12]: plot_yield_curve('2006-03-23', 'Figure 5, ')
```

```
/tmp/ipykernel_29/1226209266.py:6: UserWarning: set_ticklabels() should only be used
with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_yticklabels(['{:0.2f}%'.format(y) for y in ax.get_yticks()])
```

Figure 5, Treasury Yield Curve



In Figure 5, the yields from 6-month maturity to 10-year maturity demonstrate a downward slope shape, which is different from the yield curve at 2020-01-10. Let's see if the NS model will pick this up. We'll repeat the same process to model the yield curve on 2006-03-23.

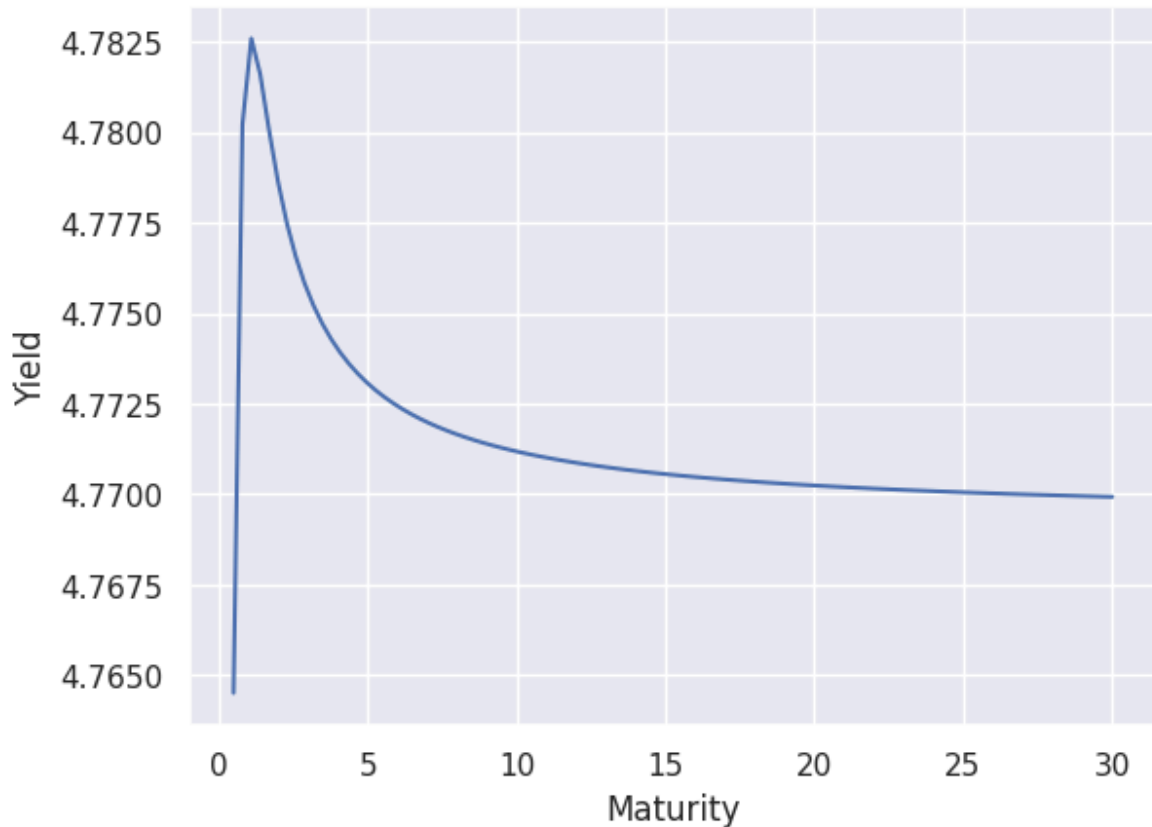
```
In [13]: y = np.array(yields.loc["2006-03-23"])
curve, status = calibrate_ns_ols(t, y, tau0=0.5) # starting value of 0.5 for the o
assert status.success
print(curve)
```

```
NelsonSiegelCurve(beta0=4.769301547470864, beta1=-0.1860856841622217, beta2=0.259371
1669247393, tau=0.2570489403547519)
```

```
In [14]: y_hat = curve
t_hat = np.linspace(0.5,30,100)
plt.plot(t_hat, y_hat(t_hat))
plt.xlabel("Maturity")
plt.ylabel("Yield")
plt.title("Figure 6, NS Model Result")
```

```
Out[14]: Text(0.5, 1.0, 'Figure 6, NS Model Result')
```

Figure 6, NS Model Result



From the above Figure 6, we can see that the curve slopes downward after one year of maturity. This result is consistent with Figure 5, which we drew earlier.

5. Cubic Spline Fitting of Yield Curve

In this section, we are going to introduce another popular method to fit a yield curve with data. It is called **cubic spline**. Spline fitting is a method under polynomial fitting. Please read the required reading "Spine Method of Interpolation" to study the theory of spline fitting, especially cubic spline fitting. We'll use the cubic spline fitting method in Python to fit a yield curve in the next section. Please be advised that in Lesson 4 "Application of Quadratic Spline Interpolation", there is a mistake in the note. In the big matrix presentation, the coefficient vector should be on the left-hand side of the equation, not the right-hand side.

5.1 Python Application: Use Cubic Spline Fitting to Fit a Yield Curve

In this section, we'll demonstrate how to fit a yield curve cubic spline. For ease of demonstration, we will only use yields from 2-year, 5-year, 10-year, and 30-year Treasury bonds on 2020-01-10 as an example. Let's check the yields on 2020-01-10 first.

```
In [15]: yields.loc["2020-01-10"]
```

```
Out[15]: 1 Month    1.52
          3 Month    1.54
          6 Month    1.55
          1 Year     1.53
          2 Year     1.56
          3 Year     1.59
          5 Year     1.63
          7 Year     1.74
          10 Year    1.83
          20 Year    2.14
          30 Year    2.28
          Name: 2020-01-10 00:00:00, dtype: float64
```

Let's define our maturity variable and yield variable as array forms.

```
In [16]: t = np.array([2,5,10,30])
          y = np.array([1.56,1.63,1.83,2.28])
```

Now, let's write out the cubic spline equations first. Since we have 4 paired data points, there will be 3 splines.

$$\begin{aligned} \text{for } 2 \leq x \leq 5: & \quad f(x) = a_1 x^3 + b_1 x^2 + c_1 x + d_1 \\ \text{for } 5 \leq x \leq 10: & \quad f(x) = a_2 x^3 + b_2 x^2 + c_2 x + d_2 \\ \text{for } 10 \leq x \leq 30: & \quad f(x) = a_3 x^3 + b_3 x^2 + c_3 x + d_3 \end{aligned}$$

From the above equations, we have 12 unknowns. Hence, we need 12 equations to solve for the 12 parameters. Let's write out the equations that each cubic spline function would go through at two consecutive data points.

$$\begin{aligned} a_1(2)^3 + b_1(2)^2 + c_1(2) + d_1 &= 1.56 \\ a_1(5)^3 + b_1(5)^2 + c_1(5) + d_1 &= 1.63 \end{aligned}$$

$$\begin{aligned} a_2(5)^3 + b_2(5)^2 + c_2(5) + d_2 &= 1.63 \\ a_2(10)^3 + b_2(10)^2 + c_2(10) + d_2 &= 1.83 \end{aligned}$$

$$\begin{aligned} a_3(10)^3 + b_3(10)^2 + c_3(10) + d_3 &= 1.83 \\ a_3(30)^3 + b_3(30)^2 + c_3(30) + d_3 &= 2.28 \end{aligned}$$

Now, let's write down the equations that show the first derivatives of two consecutive cubic splines being continuous at the common interior points.

$$\begin{aligned} 3a_1(5)^2 + 2b_1(5) + c_1 &= 3a_2(5)^2 + 2b_2(5) + c_2 \\ 3a_2(10)^2 + 2b_2(10) + c_2 &= 3a_3(10)^2 + 2b_3(10) + c_3 \end{aligned}$$

And then we'll write the equations that show the second derivatives of two consecutive cubic

splines being continuous at the common interior points.

$$6a_1(5) + 2b_1 = 6a_2(5) + 2b_2 \quad 6a_2(10) + 2b_2 = 6a_3(10) + 2b_3$$

The last two equations are boundary conditions. We set the second derivatives of the cubic splines at the end points to be zeroes.

$$6a_1(2) + 2b_1 = 0 \quad 6a_3(30) + 2b_3 = 0$$

Now we have 12 equations to solve for 12 parameters. We can write the whole problem as a big matrix equation.

$$\begin{bmatrix} 8 & 4 & 2 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 125 & 25 & 5 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1000 & 100 & 10 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 100 & 10 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 27000 & 900 & 30 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 75 & 10 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -75 & -10 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 300 & 20 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -20 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 30 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -30 & -2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 180 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} a_1 \\ b_1 \\ c_1 \\ d_1 \\ a_2 \\ b_2 \\ c_2 \\ d_2 \\ a_3 \\ b_3 \\ c_3 \\ d_3 \end{bmatrix} = \begin{bmatrix} 1.56 \\ 1.63 \\ 1.63 \\ 1.83 \\ 1.83 \\ 2.28 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

We can write the above matrix equation as the following equation:

$$A \bullet c = y$$

A is the squared matrix. c is the coefficient vector and y is the output vector. In order to solve for c , we will use the following linear algebra rule.

$$c = A^{-1} \bullet y$$

A^{-1} is the inverse of the squared matrix. The following Python code is used to solve for the coefficient vector.

```
In [17]: # Create output vector y (out variable) and squared matrix A (input variable)
out = np.array([1.56,1.63,1.63,1.83,1.83,2.28,0,0,0,0,0,0])
input = np.array([[8,4,2,1,0,0,0,0,0,0,0,0],[125,25,5,1,0,0,0,0,0,0,0,0],[0,0,0,0,1,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,1000,100,10,1],[0,0,0,0,0,0,0,0,27000,900,30,1],
[100,10,1,0,0,0,0,0,0,0,0,0],[75,10,1,0,0,0,0,0,0,0,0,0],[-75,-10,-1,0,0,0,0,0,0,0,0,0],
[300,20,1,0,0,0,0,0,0,0,0,0],[-20,-1,0,0,0,0,0,0,0,0,0,0],[30,2,0,0,0,0,0,0,0,0,0,0],
[-30,-2,0,0,0,0,0,0,0,0,0,0],[0,0,0,0,60,2,0,0,-60,-2,0,0],[12,2,0,0,0,0,0,0,0,0,0,0]])
```

```
In [18]: # Solve for coefficient vector and reshape to an 3 by 4 array (lines variable)
# Make sure to give enough decimals since all coefficients are relatively small. If
lines = np.round(np.dot(np.linalg.inv(input), out).reshape(-1,4), decimals=8)
lines
```

```
Out[18]: array([[ 3.96060000e-04, -2.37634000e-03,  2.45215100e-02,
                  1.51729391e+00],
                [-3.31400000e-04,  8.53548000e-03, -3.00376300e-02,
                  1.60822581e+00],
                [ 2.34400000e-05, -2.10968000e-03,  7.64139800e-02,
                  1.25338710e+00]])
```

From the above result, we can see the coefficients are presented as a 3 X 4 array.

$$\begin{bmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \end{bmatrix}$$

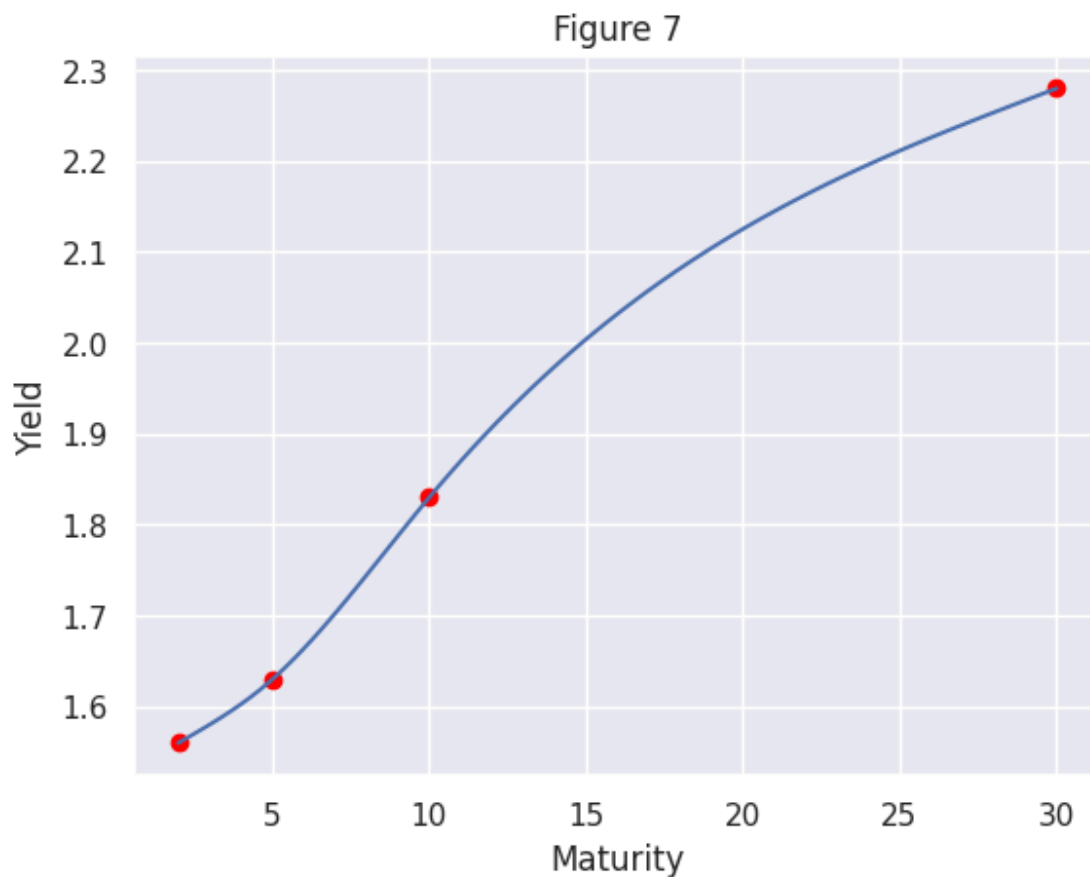
Now we can draw a smooth curve from maturity 2 to maturity 30 using Python.

```
In [19]: # Calculates x**0 + x**1 + x**2 + x**3
def plot_num(values, coeffs):
    # Coeffs are assumed to be in order 0, 1, ..., n-1
    expanded = np.hstack([coeffs[i] * (values ** i) for i in range(0, len(coeffs))])
    return np.sum(expanded, axis=1)
```

```
In [20]: # Simulate the 100 paired data points and draw the graph
xs = np.linspace(2,30, 100)

y1s = plot_num(xs[xs<5].reshape(-1,1), lines[0][::-1])
y2s = plot_num(xs[(xs>=5) & (xs<10)].reshape(-1,1), lines[1][::-1])
y3s = plot_num(xs[xs>=10].reshape(-1,1), lines[2][::-1])
ys = np.concatenate([y1s, y2s, y3s])

plt.plot(xs, ys)
plt.scatter(t, y,c="red")
plt.xlabel("Maturity")
plt.ylabel("Yield")
plt.title("Figure 7")
plt.show()
```



The above Figure 7 shows the estimated yield curve on 2020-01-10 using the cubic spline fitting method. We can see with the cubic spline that the line between two data points is not a straight line but a curve. With this estimated curve, we can calculate the yield for any maturity occurring on this curve for bond pricing or other yield analyses.

Once we build this yield curve, we can use this curve to obtain discount factors for any future cash flows. Summing all the discounted future cash flows from an asset, we can get the present value of the asset and evaluate whether the current price is overvaluing or undervaluing the asset.

6. Conclusion

In this lesson, we first learned what the risk-free rate is. Then, we calculated the volatility of U.S. Treasury bonds of different maturities. We also learned two methods of fitting a yield curve using the Nelson Siegel method and cubic spline method. We finished with a demonstration of how to implement these two methods to fit a U.S. Treasury yield curve in Python.

References

- Pape. "Understanding the Nelson-Siegel-Svensson (NSS) Model for Bond Yield Curve Analysis." Medium, 2024 May 12. <https://medium.com/@pape14/understanding-the->

[nelson-siegel-svensson-nss-model-for-bond-yield-curve-analysis-2a23202cbf6b](#).

- Svensson, Lars. "Estimating and Interpreting Forward Interest Rates: Sweden 1992-1994." *NBER Working Paper Series*, no. 4871, 1994.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Kuttler, Ken. "7.1: Eigenvalues and Eigenvectors of a Matrix." LibreTexts Mathematics.
[https://math.libretexts.org/Bookshelves/Linear_Algebra/A_First_Course_in_Linear_Algebra_\(Kuttler\)/07%3A_Spectral_Theory/7.01%3A_Eigenvalues_and_Eigenvectors_of_a_Matrix](https://math.libretexts.org/Bookshelves/Linear_Algebra/A_First_Course_in_Linear_Algebra_(Kuttler)/07%3A_Spectral_Theory/7.01%3A_Eigenvalues_and_Eigenvectors_of_a_Matrix).
- **Required pages:** "Definition to Eigenvectors and Eigenvalues" section and "Finding Eigenvectors and Eigenvalues" section
 - **Estimated reading time:** 60 minutes

GOVERNMENT BOND YIELD BIVARIATE ANALYSIS AND YIELD CURVE ANALYSIS

MODULE 1 | LESSON 4

Reading Time 60 minutes

Prior Knowledge U.S. Treasury Bonds, Yield Curve, Linear Algebra, Basic Python

Keywords Bond price-yield curve, Risk free interest rate, Nelson Siegel, Cubic Spline

In the previous lesson, we introduced U.S. Treasury price yield curve and discussed the shape of the curve. We also introduced the methods to fit U.S. Treasury curves using polynomial fitting techniques. In this lesson, we will continue to learn new analytical methods to understand U.S. Treasury yields. First, we will explore bivariate relationships of different yields, including correlation and covariance. Then, we will learn a new technique to extract key features of the U.S. Treasury yield curve to analyze the yield curve's behavior.

1. Bivariate Analysis

Bivariate analysis is a collection of methods to analyze the relationship between two variables. In finance, we are particularly interested in understanding how two variables interact with each other. In this lesson, we'll look at the yields of different U.S. Treasury bonds and how they move in relation to each other. The first bivariate analysis tool we will learn is covariance.

1.1 Covariance

Covariance is a metric to measure how two variables move together. Specifically, covariance calculates the amount of movement the two variables exhibit. Here is the covariance formula:

$$\text{Cov}(X,Y)=E[(X-E[X])(Y-E[Y])]$$

Here are some of the properties of covariance:

1. If the covariance has a positive sign, it means the two variables move in the same direction.
2. If the covariance has a negative sign, the two variables move in opposite directions.
3. If the covariance is 0, the two variables are linearly uncorrelated (uncorrelated).

The higher the absolute value of the covariance of the two variables, the stronger the (positive or negative) relationship the two variables have. Let's pull some U.S. Treasury yield data to demonstrate covariance between two different yields.

A covariance matrix is a bivariate analysis measure for two variables. However, when we have several variables and want to know the pairwise relationships of these variables, we'll need a covariance matrix. A **covariance matrix** is a square matrix that represents the pairwise covariances between multiple variables in a dataset. Now, let's pull U.S. Treasury yield data and investigate their covariance matrix.

```
In [1]: import pandas as pd
        from fredapi import Fred
        import matplotlib.pyplot as plt
        import seaborn as sns
        sns.set()
```

```
In [2]: # Initialize the FRED API with your key
        fred = Fred(api_key='5079f41d061a4037d81f3da69e018803') # Replace my APIKEY with "Y

        # List of Treasury yield series IDs
        series_ids = ['DGS1M0', 'DGS3M0', 'DGS6M0', 'DGS1', 'DGS2', 'DGS3', 'DGS5', \
                      'DGS7', 'DGS10', 'DGS20', 'DGS30']

        # Function to get data for a single series
        def get_yield_data(series_id):
            data = fred.get_series(series_id, observation_start="1975-01-01", observation_e

            return data

        # Get data for all series
        yields_dict = {series_id: get_yield_data(series_id) for series_id in series_ids}

        # Combine into a single DataFrame
        yields = pd.DataFrame(yields_dict)

        # Rename columns for clarity
        yields.columns = ['1 Month', '3 Month', '6 Month', '1 Year', '2 Year', '3 Year', '5
                          '7 Year', '10 Year', '20 Year', '30 Year']
```

```
In [3]: # Make datetime as the index
        yields.index = pd.to_datetime(yields.index)
```

```
In [4]: # Drop NaN in the dataset
        yields = yields.dropna()
```

```
In [5]: # Calculate covariance matrix for US Treasury yields in the dataset
        covariance_matrix = yields.cov()
        print("Covariance Matrix:")
        print(covariance_matrix)
```

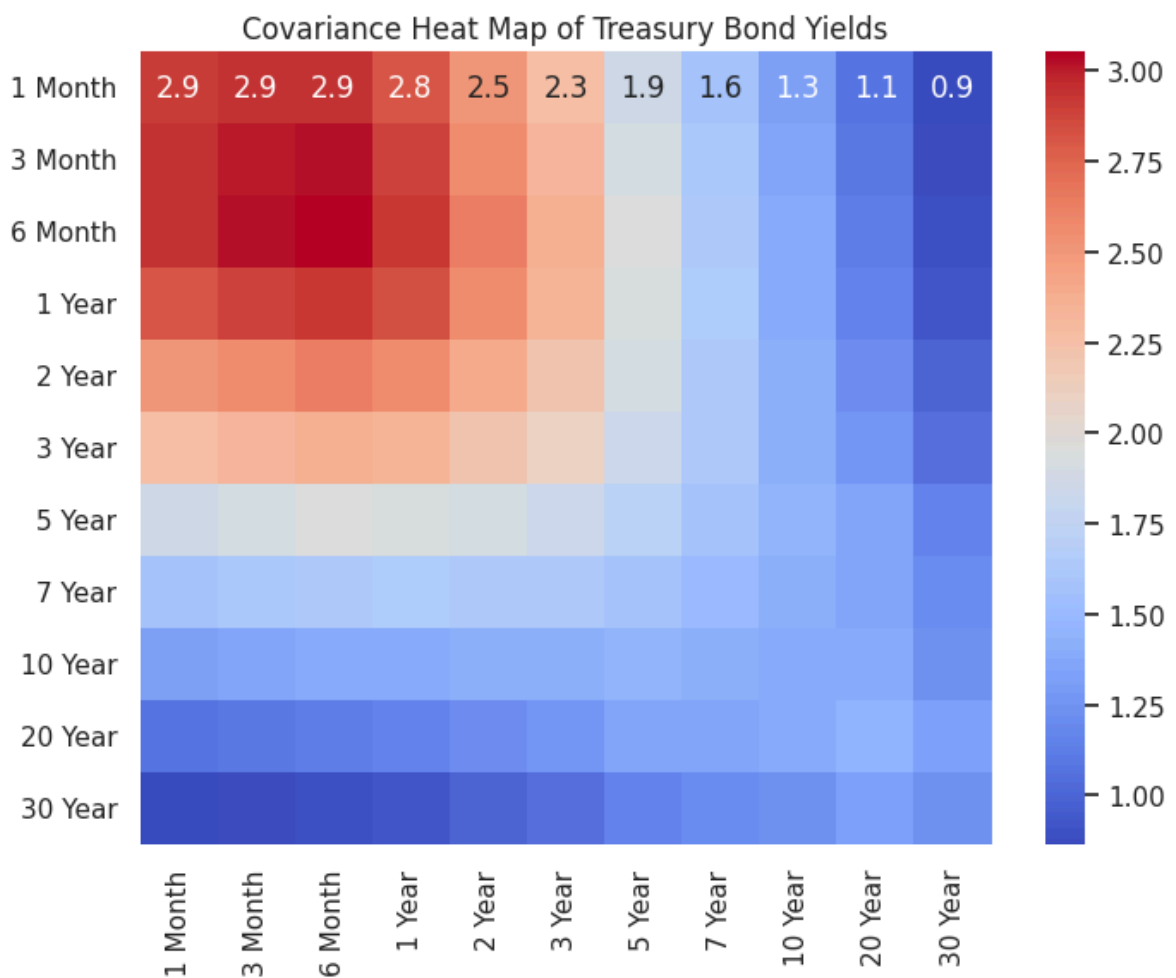
Covariance Matrix:

	1 Month	3 Month	6 Month	1 Year	2 Year	3 Year	5 Year	\
1 Month	2.909927	2.946278	2.948709	2.812408	2.499031	2.251515	1.850674	
3 Month	2.946278	3.004493	3.018967	2.887166	2.571375	2.317856	1.901841	
6 Month	2.948709	3.018967	3.051413	2.931246	2.624882	2.372696	1.951042	
1 Year	2.812408	2.887166	2.931246	2.836161	2.569272	2.340278	1.944752	
2 Year	2.499031	2.571375	2.624882	2.569272	2.388302	2.216908	1.897526	
3 Year	2.251515	2.317856	2.372696	2.340278	2.216908	2.092766	1.842915	
5 Year	1.850674	1.901841	1.951042	1.944752	1.897526	1.842915	1.714385	
7 Year	1.565717	1.605385	1.647697	1.654011	1.648671	1.634697	1.582433	
10 Year	1.315260	1.344596	1.381242	1.395294	1.418752	1.435157	1.447139	
20 Year	1.078020	1.098030	1.128773	1.154327	1.214425	1.267258	1.350980	
30 Year	0.862766	0.872645	0.893530	0.919017	0.988536	1.052167	1.161403	

	7 Year	10 Year	20 Year	30 Year
1 Month	1.565717	1.315260	1.078020	0.862766
3 Month	1.605385	1.344596	1.098030	0.872645
6 Month	1.647697	1.381242	1.128773	0.893530
1 Year	1.654011	1.395294	1.154327	0.919017
2 Year	1.648671	1.418752	1.214425	0.988536
3 Year	1.634697	1.435157	1.267258	1.052167
5 Year	1.582433	1.447139	1.350980	1.161403
7 Year	1.504196	1.416231	1.371829	1.204537
10 Year	1.416231	1.378491	1.383803	1.238816
20 Year	1.371829	1.383803	1.455676	1.324389
30 Year	1.204537	1.238816	1.324389	1.229380

From the above covariance matrix for U.S. Treasury yields, we can see all pairwise yields have positive covariances. It means when the yield of one maturity increases, the other yields will also increase. We can visualize the covariance matrix with a heatmap in Python.

```
In [6]: #Make a heatmap for covariance matrix
plt.figure(figsize=(8, 6))
sns.heatmap(covariance_matrix, annot=True, cmap='coolwarm', fmt=".1f")
plt.title('Covariance Heat Map of Treasury Bond Yields')
plt.show()
```



From the above covariance matrix heatmap, we can better read the differences of the covariance numbers. However, one downside of covariance is its value changes when the scales of two variables change. We cannot just compare different covariances for different pairwise variables to evaluate how close they move together. Because of this issue, we use correlation more often in data analysis.

1.2 Correlation

Correlation is also a metric to measure the co-movement of the two variables. However, it eliminates the scale issue mentioned above by dividing covariance with the square root of the multiplication of the two variables' variances. Here is the correlation math formula:

$$\text{Corr}(X,Y)=\frac{\text{Cov}(X,Y)}{\sqrt{\text{Var}(X)*\text{Var}(Y)}}$$

where $\text{Cov}(X,Y)$ is the covariance of X and Y , $\text{Var}(X)$ and $\text{Var}(Y)$ are variances of X and Y .

Here are some properties of correlation:

1. Unlike covariance, the value of correlation is limited between -1 and 1 .

2. If the correlation of two variables is greater than 0, the two variables are positively correlated.
3. If the correlation of two variables is less than 0, the two variables are negatively correlated.
4. If two variables are perfectly positively correlated, the correlation will be 1.
5. If two variables are perfectly negative correlated, the correlation will be -1 .
6. If the correlation is 0, the two variables are linearly uncorrelated.

Like with a covariance matrix, if we have several variables, we can use a **correlation matrix** to present correlation metrics for pairwise variables. Let's calculate the correlation matrix for our U.S. Treasury yield data.

```
In [7]: # Calculate correlation matrix for US Treasury yields in the dataset
correlation_matrix = yields.corr()
print("Correlation Matrix:")
print(correlation_matrix)
```

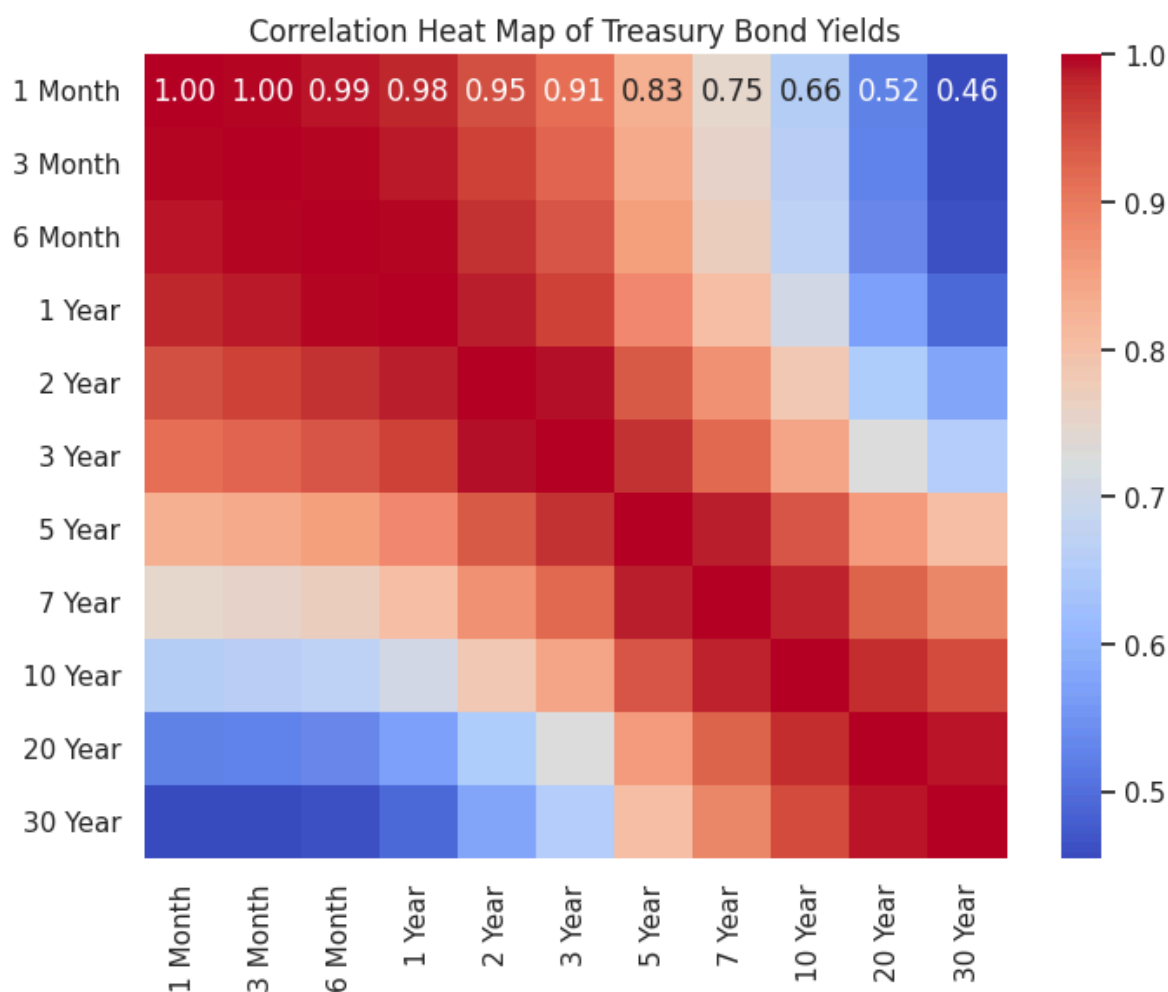
Correlation Matrix:

	1 Month	3 Month	6 Month	1 Year	2 Year	3 Year	5 Year	
1 Month	1.000000	0.996431	0.989556	0.978975	0.947951	0.912375	0.828580	\
3 Month	0.996431	1.000000	0.997062	0.989056	0.959920	0.924359	0.837981	
6 Month	0.989556	0.997062	1.000000	0.996406	0.972332	0.938926	0.853025	
1 Year	0.978975	0.989056	0.996406	1.000000	0.987188	0.960598	0.881951	
2 Year	0.947951	0.959920	0.972332	0.987188	1.000000	0.991614	0.937754	
3 Year	0.912375	0.924359	0.938926	0.960598	0.991614	1.000000	0.972950	
5 Year	0.828580	0.837981	0.853025	0.881951	0.937754	0.972950	1.000000	
7 Year	0.748376	0.755164	0.769085	0.800794	0.869836	0.921350	0.985414	
10 Year	0.656702	0.660700	0.673469	0.705664	0.781916	0.844962	0.941356	
20 Year	0.523785	0.525045	0.535579	0.568108	0.651319	0.726060	0.855190	
30 Year	0.456151	0.454056	0.461334	0.492170	0.576905	0.655966	0.799992	

	7 Year	10 Year	20 Year	30 Year
1 Month	0.748376	0.656702	0.523785	0.456151
3 Month	0.755164	0.660700	0.525045	0.454056
6 Month	0.769085	0.673469	0.535579	0.461334
1 Year	0.800794	0.705664	0.568108	0.492170
2 Year	0.869836	0.781916	0.651319	0.576905
3 Year	0.921350	0.844962	0.726060	0.655966
5 Year	0.985414	0.941356	0.855190	0.799992
7 Year	1.000000	0.983513	0.927076	0.885778
10 Year	0.983513	1.000000	0.976877	0.951616
20 Year	0.927076	0.976877	1.000000	0.990011
30 Year	0.885778	0.951616	0.990011	1.000000

Next, let's make a heatmap for the correlation matrix to better read the result.

```
In [8]: # Make a heatmap for correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heat Map of Treasury Bond Yields')
plt.show()
```



From the above correlation matrix heatmap for U.S. Treasury yields, we can compare correlations among different pairwise yields. First, we can see the correlations on the diagonal cells are all 1s. It is easy to understand because all yields move perfectly in the same direction with themselves. Secondly, another interesting observation is that two yields that have close maturities have higher correlations. As the difference of the maturities of two yields widens, the correlation of the two yields gets lower.

Due to its easy-to-understand property, a correlation metric is a commonly used and crucial measurement for bivariate analysis. Understanding the correlation between different Treasury bond yields is crucial for risk management, portfolio construction, and understanding market dynamics. It helps quantify how different parts of the yield curve move in relation to each other, which is essential for strategies like yield curve trades and immunization in fixed-income portfolios.

2. Feature Extractions and Principal Component Analysis

In this section, we are going to introduce a method to extract key features from a dataset. It is called **principal component analysis (PCA)**. Oftentimes, we try to discover key common factors or features that can explain the movement of all variables in a dataset. If we can find

these key common factors that can represent most of the data variation in the dataset, we won't need to use the whole dataset for analysis. Instead, we can focus on analyzing these key factors. This is a data dimension reduction technique. It is very crucial that it can reduce the size of the dataset to make a lot of algorithms run more efficiently.

In this section, we will continue to use the U.S. Treasury yield dataset to demonstrate this technique. We will show a step-by-step process for conducting PCA. Our first step is to standardize the scales of variables in our dataset.

2.1 Standardizing Variables

Standardizing (or normalizing) a variable is a common statistical technique used to transform a variable into a standard scale. It converts a variable so that it has a mean of 0 and a standard deviation of 1. Here is the formula to standardize a variable:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

Z = standardized variable with mean = 0 and standard deviation = 1

X = original variable

μ = mean of the original variable

σ = standard deviation of the original variable

Here are the benefits of standardizing variables:

1. Making all variables comparable on the same scale
2. Facilitating calculation of the other analyses requiring same-scale variables

Now let's standardize the U.S. Treasury yield dataset. We first need to calculate the mean and standard deviation of each yield. Then, we will apply the above formula to our dataset to create a standardized (normalized) dataset.

```
In [9]: #Calculate means for all yields in the dataset
yield_means = yields.mean()
print("Yield Means:")
print(yield_means)
```

Yield Means:

```
1 Month    1.449053
3 Month    1.518396
6 Month    1.627451
1 Year      1.717973
2 Year      1.905475
3 Year      2.097270
5 Year      2.485802
7 Year      2.806259
10 Year     3.088349
20 Year     3.620395
30 Year     3.724571
dtype: float64
```

```
In [10]: #Calculate standard deviations for all yields in the dataset
yield_stds = yields.std()
print("Yield Standard Deviations:")
print(yield_stds)
```

Yield Standard Deviations:

```
1 Month    1.705851
3 Month    1.733347
6 Month    1.746829
1 Year      1.684091
2 Year      1.545413
3 Year      1.446639
5 Year      1.309345
7 Year      1.226457
10 Year     1.174092
20 Year     1.206514
30 Year     1.108774
dtype: float64
```

```
In [11]: # Now create a standardized US Treasury yield dataset
standardized_data = (yields - yield_means) / yield_stds
print("Standardized Yield (first 5 rows):")
print(standardized_data.head())
```

Standardized Yield (first 5 rows):

	1 Month	3 Month	6 Month	1 Year	2 Year	3 Year	
2001-07-31	1.301958	1.166300	1.054796	1.075968	1.219431	1.356751	\
2001-08-01	1.290234	1.160531	1.054796	1.093781	1.245314	1.377489	
2001-08-02	1.290234	1.160531	1.049071	1.099719	1.284139	1.432789	
2001-08-03	1.278510	1.154762	1.054796	1.099719	1.297080	1.467352	
2001-08-06	1.272647	1.154762	1.054796	1.093781	1.277668	1.432789	
	5 Year	7 Year	10 Year	20 Year	30 Year		
2001-07-31	1.591786	1.674532	1.687817	1.649052	1.610273		
2001-08-01	1.629973	1.707147	1.721886	1.665629	1.628311		
2001-08-02	1.683435	1.764222	1.772989	1.707071	1.664387		
2001-08-03	1.706348	1.780529	1.798541	1.723647	1.682425		
2001-08-06	1.698710	1.780529	1.790023	1.723647	1.682425		

Usually, preparing a standardized dataset is part of the data preparation step before running the analysis. Make sure to standardize your dataset if the analysis you are going to run requires variables to be on the same scale.

Before moving on to the next step of the PCA, we need to learn new tools from linear algebra. In the next two sections, we will introduce eigenvectors and eigenvalues.

2.2 Eigenvectors and Eigenvalues

In this section, we will learn the definitions of eigenvectors and eigenvalues and how to calculate them. Eigenanalysis is a very important linear algebra method in finance. Please review the required reading for this lesson to understand this topic. In the next section, we will use visualization to enhance our understanding of eigenvectors and eigenvalues.

2.3 Visualization of Eigenvectors and Eigenvalues

In this section, we will use graphs to explain the concepts of eigenvectors and eigenvalues.

Vectors in a two-dimensional coordinate system are described by their magnitude and direction. A linear transformation occurs when we multiply a vector by a matrix. The transformation can change both the vector's magnitude and direction. However, there are certain unique vectors that will maintain their direction or just switch to the opposite direction when a linear transformation happens. The only change is the magnitude of these vectors. These special vectors are called eigenvectors. Let's first revisit the equation between eigenvectors and eigenvalues from the last equation:

$$Ax = \lambda x$$

where:

A is the linear transformation matrix

x is eigenvector

λ is the magnitude metric or eigenvalue

Let's first look at the left side of the equation. It corresponds to the linear transformation of a vector we described above. On the right side, the vector multiplies with a scalar. When the left-hand side of the equation equals the right-hand side of the equation, it means the linear transformation of the vector equals a magnitude change of the same vector. Based on what we described about an eigenvector, the vector in this equation is an eigenvector. Figure 1 below demonstrates this concept visually.

Figure 1: Visual Demonstration of Eigenvectors and Eigenvalues

 Graph showing a linear transformation of an (x, y) system with a circle transformed into an ellipse, highlighting eigenvector v_1 and its scaled version λv_1

From the above figure, we can see the purple eigenvector preserves its direction after the linear transformation, which is shown by the orange vector. The transformation merely stretches or compresses the eigenvector, without changing its direction.

The eigenvalue associated with an eigenvector is the number by which the eigenvector is scaled during transformation.

- If the eigenvalue is positive, the eigenvector is stretched in its original direction.
- If negative, the eigenvector is stretched in the opposite direction.
- If the eigenvalue is 1, the eigenvector remains unchanged.
- If 0, the eigenvector is collapsed to a point. When representing eigenvalues as vector lengths, the length of each eigenvector arrow can be scaled to represent its corresponding eigenvalue.

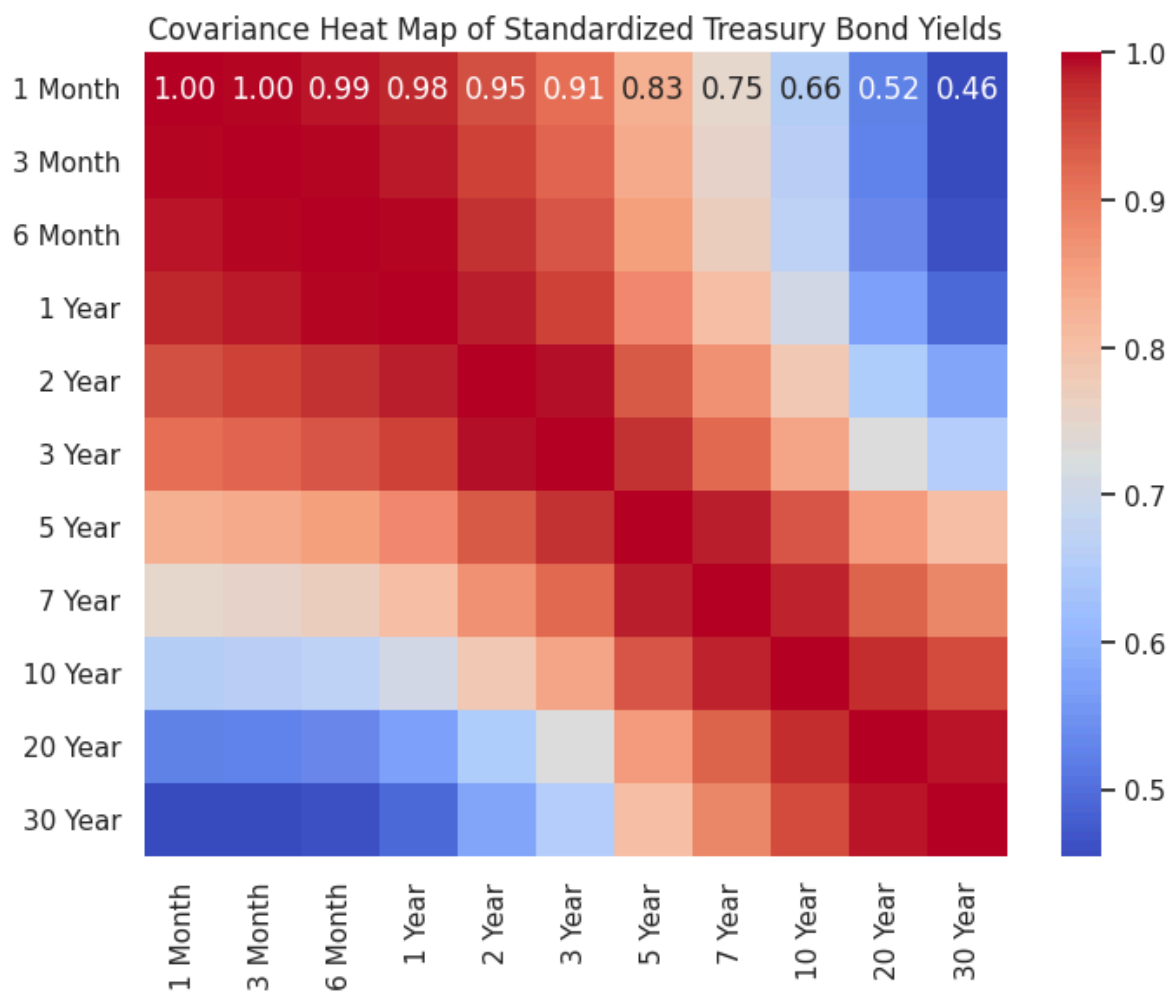
2.4 Derive Principal Components

The next step of PCA is to find the covariance matrix of the standardized dataset. For a standardized dataset, its covariance matrix will be the same as the correlation matrix of the pre-standardized dataset. In the following Python code, we first import the necessary packages for matrix manipulation and for eigenvector/eigenvalue calculation. Then, we will get the covariance matrix for the standardized data and draw a heatmap for the covariance matrix.

```
In [12]: import numpy as np
        from numpy import linalg as LA
```

```
In [13]: # Calculate covariance matrix of the standardized dataset
        std_data_cov = standardized_data.cov()
```

```
In [14]: # Draw a heatmap of the covariance matrix
        plt.figure(figsize=(8, 6))
        sns.heatmap(std_data_cov, annot=True, cmap='coolwarm', fmt=".2f")
        plt.title('Covariance Heat Map of Standardized Treasury Bond Yields')
        plt.show()
```



Now let's calculate the eigenvectors and eigenvalues of the covariance matrix of the standardized yield dataset.

```
In [15]: # Calculate eigenvectors and eigenvalues of the covariance matrix of standardized y
eigenvalues, eigenvectors = LA.eig(std_data_cov)
eigenvalues
```

```
Out[15]: array([9.22236235e+00, 1.63341019e+00, 1.17347718e-01, 1.46747161e-02,
 5.37172584e-03, 3.70149555e-03, 1.61309240e-03, 6.97458739e-04,
 4.12103724e-04, 2.47130907e-04, 1.62022336e-04])
```

```
In [16]: eigenvectors
```

```
Out[16]: array([[ 0.29838306,  0.30407626,  0.44114252, -0.53274513,  0.16709103,
                  0.3702677 ,  0.33473909, -0.24039038,  0.02716335, -0.06687106,
                  -0.00401581],
                [ 0.3004742 ,  0.30709453,  0.3256805 , -0.10407045, -0.09017425,
                  -0.13428768, -0.45363295,  0.58333199, -0.13905681,  0.31674937,
                  0.08135824],
                [ 0.30335843,  0.29873569,  0.17614053,  0.25730089, -0.24045099,
                  -0.34295333, -0.10807462, -0.13460411,  0.32846299, -0.58367918,
                  -0.26019232],
                [ 0.30894871,  0.26727645, -0.00945148,  0.42112655, -0.09742097,
                  -0.16475673,  0.15471834, -0.48093818, -0.17006916,  0.43202651,
                  0.38500876],
                [ 0.31879358,  0.17616334, -0.28944958,  0.31333639,  0.25331718,
                  0.2316792 ,  0.23709616,  0.21742618, -0.3594238 ,  0.02750551,
                  -0.57801452],
                [ 0.32383758,  0.08574516, -0.41253571,  0.06981027,  0.2557664 ,
                  0.27857918, -0.04283274,  0.253605 ,  0.24613993, -0.32221553,
                  0.58235483],
                [ 0.32393331, -0.09082567, -0.38335179, -0.28984745,  0.02440707,
                  -0.0335848 , -0.25924506, -0.22190129,  0.51816781,  0.41667104,
                  -0.30913315],
                [ 0.31475338, -0.2155924 , -0.25536488, -0.38788729, -0.13245802,
                  -0.19980121, -0.25664372, -0.23752598, -0.60974821, -0.28958861,
                  0.05811859],
                [ 0.29830301, -0.32947806, -0.0224116 , -0.14078023, -0.27752025,
                  -0.36651204,  0.64790245,  0.35934669,  0.10798909,  0.07495589,
                  0.06783763],
                [ 0.2677569 , -0.44896794,  0.22584655,  0.25821905, -0.48820444,
                  0.58883593, -0.14816743, -0.03038806,  0.02106533,  0.00408296,
                  -0.02532805],
                [ 0.24905042, -0.49930724,  0.38802724,  0.19437911,  0.65844796,
                  -0.21614344, -0.11702391, -0.07221199,  0.03078012, -0.01338036,
                  0.00323551]])
```

From the above output, we can see that the eigenvalues and corresponding eigenvectors are ordered in descending order by the values of eigenvalues. In PCA, we also call eigenvectors **loadings**. We will see why this is important later.

We can view this collection of all eigenvectors as a linear transformation matrix to the standardized dataset. The transformed data will have a very interesting feature that we will introduce soon. Let's transform the standardized data with eigenvectors first.

```
In [17]: # Transform standardized data with Loadings
principal_components = standardized_data.dot(eigenvectors)
principal_components.columns = ["PC_1", "PC_2", "PC_3", "PC_4", "PC_5", "PC_6", "PC_7", "P
principal_components.head()
```

Out[17]:

	PC_1	PC_2	PC_3	PC_4	PC_5	PC_6	PC_7	PC_8	
2001-07-31	4.608202	-0.918184	0.138746	-0.223360	0.013690	0.063247	0.008572	0.006278	0.
2001-08-01	4.665170	-0.950595	0.102492	-0.220182	0.013318	0.054096	0.014505	0.002266	0.
2001-08-02	4.766161	-1.009754	0.054519	-0.230244	0.021179	0.064157	0.017119	0.011726	-0.
2001-08-03	4.807092	-1.038602	0.027695	-0.224234	0.025575	0.063723	0.018644	0.020406	0.
2001-08-06	4.781112	-1.044855	0.048161	-0.228694	0.013594	0.051783	0.009102	0.010319	0.

The above result shows 11 transformed variables. In PCA, they are called **principal components**. Remember we mentioned earlier that the eigenvalues from PCA are in descending order. These principal components are also presented in the same order as their corresponding eigenvalues. For example, PC_1 corresponds to the first eigenvalue, which is 9.22. PC_2 corresponds to the second highest eigenvalue, which is 1.63.

The most important feature of PCA is **the leading principal components can explain higher portions of the variance of the dataset than the rest of the principal components**. For example, PC_1 can explain more variance in the standardized data than PC_2. But by how much? The corresponding eigenvalue for PC_1 is the variance of the whole data explained by PC_1. It is 9.22 in this case. By the same logic, PC_2 explains 1.63 of the total variance of the dataset. We can sum up all the eigenvalues to get the total variance of the data. From the total variance of the data, we can also calculate the percentage of variance contribution each principal component catches. Here is the Python code.

```
In [18]: # Put data into a DataFrame
df_eigval = pd.DataFrame({"Eigenvalues":eigenvalues}, index=range(1,12))

# Work out explained proportion
df_eigval["Explained proportion"] = df_eigval["Eigenvalues"] / np.sum(df_eigval["Eigenvalues"])
#Format as percentage
df_eigval.style.format({"Explained proportion": "{:.2%}"})
```

Out[18]:

	Eigenvalues	Explained proportion
1	9.222362	83.84%
2	1.633410	14.85%
3	0.117348	1.07%
4	0.014675	0.13%
5	0.005372	0.05%
6	0.003701	0.03%
7	0.001613	0.01%
8	0.000697	0.01%
9	0.000412	0.00%
10	0.000247	0.00%
11	0.000162	0.00%

From the above table, we can see PC_1 can explain almost 84% of the variance in the standardized data. PC_2 can explain almost 15% of the variance in the standardized data. The first two leading principal components can explain almost 99% of the variance in the dataset. The rest of the 9 principal components only explain 1% of the variance of the data. Hence, to make data analysis more efficient without losing too much information, we can just use the first two principal components for analysis instead of all 11 principal components. Due to this special feature, we also call PCA a data-dimension reduction technique.

PCA's application is very wide in financial analysis and machine learning. For example, for image analysis, the original collection of images may be large, but by using PCA, we can reduce the data size while still preserving the key features of the images.

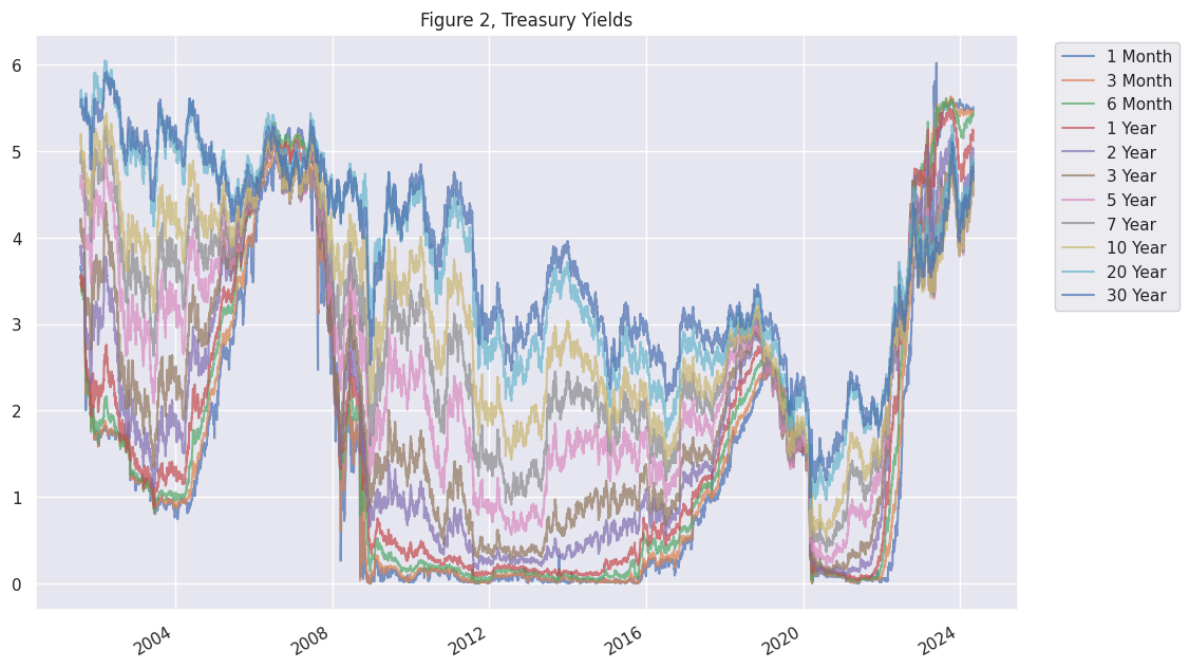
2.5. Principal Component Analysis for U.S. Treasury Yield

The great thing about using PCA to analyze the U.S. Treasury yield curve is that it can decompose the yield curve into a number of principal components (Oprea). The first three leading principal components describe the following features of the yield curve:

1. PC_1: the parallel shift of the yield curve (shift).
2. PC_2: the flattening or steeping of the yield curve (tilt).
3. PC_3: Curvature change of the yield curve (twist).

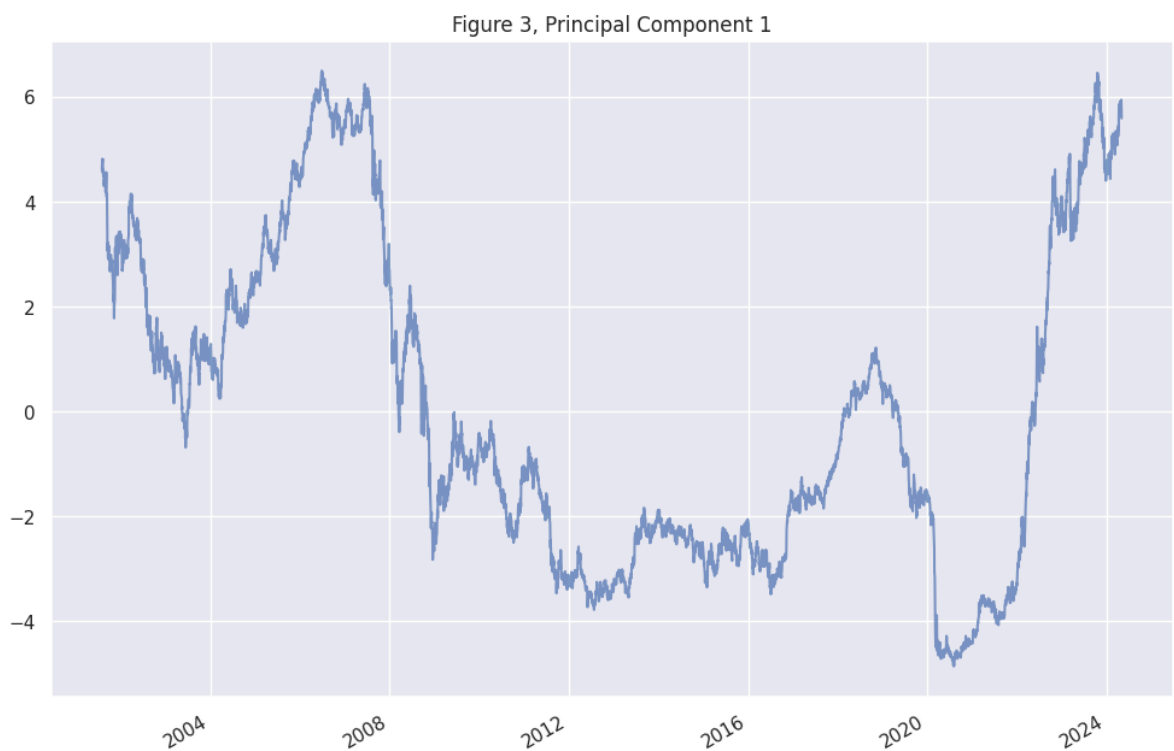
First, let's draw the yield curve as the benchmark for better comparison (Bjerring).

```
In [19]: # Treasury Yield Curve
yields.plot(figsize=(12, 8), title='Figure 2, Treasury Yields', alpha=0.7) # Plot t
plt.legend(bbox_to_anchor=(1.03, 1))
plt.show()
```



Now, let's draw principal components. Let's start from PC_1.

```
In [20]: # Plot PC_1
principal_components["PC_1"].plot(figsize=(12, 8), title='Figure 3, Principal Compo
plt.show()
```



From the above Figure 3 of PC_1, we can see the pattern looks similar to the patterns of Figure 2 (this graph starts from 2001 because we dropped NaN). This PC_1 is used to represent the yield curve parallel shift. Next, let's analyze PC_2. PC_2 is usually used to analyze how tilted the yield curve is. The following demonstration in Python code explains why PC_2 is used to analyze tilt.

First, let's calculate the difference of 2-year yield and 10-year yield as the yield curve tilt and draw a graph.

```
In [21]: #Calculate slope (difference) of 2-year Treasury yield and 10-year Treasury yield
df_s = pd.DataFrame(data = standardized_data)
df_s = df_s[["2 Year", "10 Year"]]
df_s["Tilt"] = df_s["2 Year"] - df_s["10 Year"]
df_s.head()
```

```
Out[21]:
```

	2 Year	10 Year	Tilt
2001-07-31	1.219431	1.687817	-0.468386
2001-08-01	1.245314	1.721886	-0.476571
2001-08-02	1.284139	1.772989	-0.488850
2001-08-03	1.297080	1.798541	-0.501460
2001-08-06	1.277668	1.790023	-0.512355

```
In [22]: # Draw the graph of Slope of 2-Year Treasury Yield - 10-Year Treasury Yield
df_s["Tilt"].plot(figsize=(12, 8), title='Figure 4, Tilt of 2-Year Treasury Yield - 
plt.show()
```

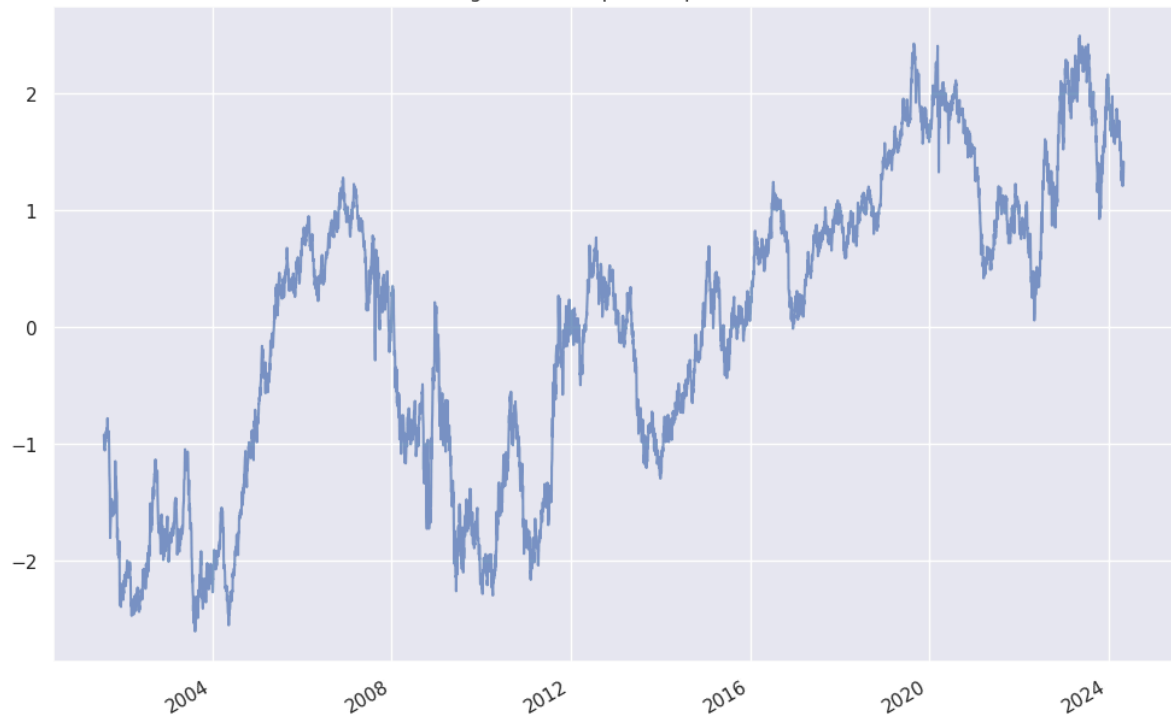
Figure 4, Tilt of 2-Year Treasury Yield - 10-Year Treasury Yield



After drawing Figure 4 for the yield difference between the 2-year Treasury yield and 10-year Treasury yield, let's draw a graph for PC_2.

```
In [23]: # Draw the graph for PC_2
principal_components["PC_2"].plot(figsize=(12, 8), title='Figure 5, Principal Component 2')
plt.show()
```

Figure 5, Principal Component 2



From Figure 4 and Figure 5, we can see that the tilt of the 2-year Treasury yield and 10-year Treasury yield graph has a pattern similar to the PC_2 graph. Let's calculate the correlation to confirm this observation.

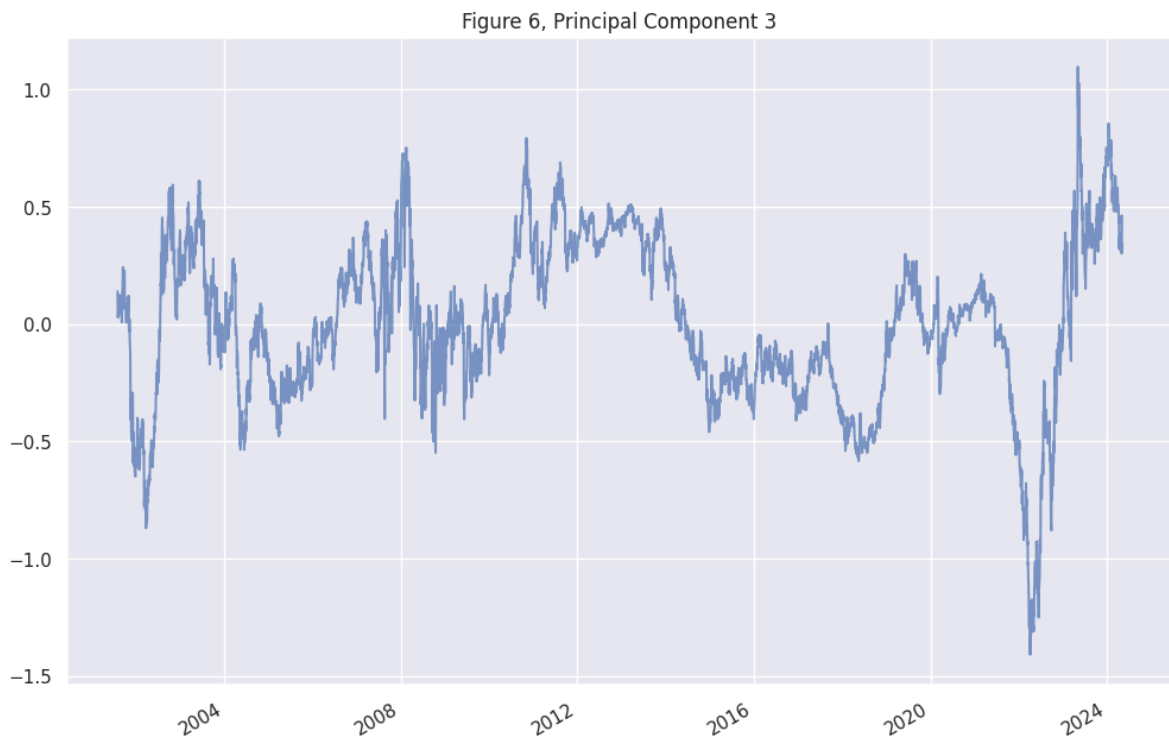
```
In [24]: np.corrcoef(principal_components["PC_2"], df_s["Tilt"])
```

```
Out[24]: array([[1.          , 0.97850538],  
               [0.97850538, 1.          ]])
```

The above code shows that the correlation of PC_2 and Tilt is 97%, which is very high. Hence, we can use the PC_2 as a proxy to analyze how tilted the yield curve is.

Now let's draw PC_3, the change of curvature of the yield curve.

```
In [25]: # Draw the graph for PC_3  
principal_components["PC_3"].plot(figsize=(12, 8), title='Figure 6, Principal Component 3',  
plt.show())
```



From Figure 6, we can see that the change in curvature of the yield curve oscillates around 0.

3. Value at Risk for a Fixed-Income Portfolio

In this section, we are going to use the feature extraction method we learned in the last section to calculate the Value at Risk of a bond portfolio.

3.1 Value at Risk (VaR)

Value at Risk (VaR) is a statistical metric to measure the potential maximum loss of an investment or a portfolio at a given time period under certain a confidence level. For example, when a stock portfolio has a VaR of \$1 million during a day at 95% confidence level, it means that there is 95% probability that this stock portfolio will not lose over \$1 million in a day.

VaR is an easy-to-understand risk metric and can be used to compare risks across different asset classes. VaR is usually used in risk management for portfolio management or regulatory reporting. It is also part of the metrics to set risk cap for traders. VaR can also be applied for capital allocation.

3.2 VaR for a Simple Treasury Bond Portfolio

We will demonstrate how to calculate VaR for a simple Treasury bond portfolio in this section. This simple bond portfolio will only consist of 2-year Treasury bonds, 5-year Treasury bonds, and 10-year Treasury bonds. First, let's create a dataset with the yields of these three bonds and calculate the daily yield percentage change in these bonds.

```
In [26]: # Create a dataset with 3 Treasury bond yields and calculate the yield changes
var_dataset = yields[["2 Year", "5 Year", "10 Year"]]
var_yield_chng_dataset = var_dataset.pct_change()
var_yield_chng_dataset = var_yield_chng_dataset.dropna()
var_yield_chng_dataset.head()
```

```
Out[26]:
```

	2 Year	5 Year	10 Year
2001-08-01	0.010554	0.010941	0.007890
2001-08-02	0.015666	0.015152	0.011742
2001-08-03	0.005141	0.006397	0.005803
2001-08-06	-0.007673	-0.002119	-0.001923
2001-08-07	0.005155	0.002123	0.001927

Next, we will prepare the dataset. We need to standardize the dataset first.

```
In [27]: # Standardize the dataset
var_yield_chng_dataset_means = var_yield_chng_dataset.mean()
var_yield_chng_dataset_stds = var_yield_chng_dataset.std()
var_yld_chng_stnd_data = (var_yield_chng_dataset - var_yield_chng_dataset_means) /
```

Now we can calculate the eigenvectors and eigenvalues of the standardized dataset.

```
In [28]: # Calculate eienvectors and eigenvalues and rank by eigenvalues
var_cov_matrix = var_yld_chng_stnd_data.cov()
eigenvalues, eigenvectors = np.linalg.eig(var_cov_matrix)
```

```
sorted_indices = np.argsort(eigenvalues)[::-1]
pca_components = eigenvectors[:, sorted_indices]
```

Let's check eigenvalues and how much variance of the data is explained by an eigenvector.

```
In [29]: # Put data into a DataFrame
df_eigval = pd.DataFrame({"Eigenvalues": eigenvalues}, index=range(1,4))

# Work out explained proportion
df_eigval["Explained proportion"] = df_eigval["Eigenvalues"] / np.sum(df_eigval["Eigenvalues"])
# Format as percentage
df_eigval.style.format({"Explained proportion": "{:.2%}"})
```

```
Out[29]:
```

	Eigenvalues	Explained proportion
1	2.545146	84.84%
2	0.384763	12.83%
3	0.070091	2.34%

From the above table, we can see the first two eigenvectors account for 97% of the variance in the dataset. Hence, we are going to select the first two eigenvectors for analysis.

```
In [30]: # Choose number of components (e.g., 2)
n_components = 2
selected_components = pca_components[:, :n_components]
```

Next, let's assume that our simple bond portfolio consists of \$2 million in 2-year Treasury bonds, \$2 million in 5-year Treasury bonds, and \$1 million in 10-year Treasury bonds.

```
In [31]: # Define a simple portfolio
portfolio = {
    2: 2000000, # $2M in 2-year bond
    5: 2000000, # $2M in 5-year bond
    10: 1000000 # $1M in 10-year bond
}
```

Next, we will calculate bond sensitivities in the portfolio. We assume the bond durations are the same as their maturity for simplicity. Then, we can calculate the portfolio value changes and VAR.

```
In [32]: # Calculate portfolio sensitivities (assuming duration = maturity for simplicity)
sensitivities = np.array([maturity * amount for maturity, amount in portfolio.items])

# Calculate portfolio value changes
portfolio_changes = (var_yield_chng_dataset * sensitivities) @ selected_components

# Calculate VaR
confidence_level = 0.95 # 95% VaR
var = -np.percentile(portfolio_changes, 100 * (1 - confidence_level))
```

```
print(f"1-day 95% VaR: ${var:,.2f}")

# Display summary statistics
print("\nSummary Statistics:")
print(f"Portfolio Value: ${sum(portfolio.values()):,.2f}")
print(f"VaR as % of Portfolio Value: {var / sum(portfolio.values()) * 100:.3f}%")
```

1-day 95% VaR: \$458,248.59

Summary Statistics:

Portfolio Value: \$5,000,000.00

VaR as % of Portfolio Value: 9.165%

The above result shows that the 1-day VaR at 95% confidence level for our simple Treasury bond portfolio is \$458,249. It is about 9% of the total portfolio value. The above example demonstrates how to use the feature extraction method to reduce the portfolio dataset and use the smaller dataset to calculate VaR.

4. Conclusion

In this lesson, we first went through the basics of bivariate analysis. We explained what bivariate analysis is and then we introduced the concepts of covariance and correlation. We then moved on to applying the feature extraction method to analyze the Treasury yield curve. We learned how to standardize a dataset. We also learned what eigenvectors and eigenvalues are. Then, we moved to conduct feature extraction from the Treasury bond yield. Next, we also applied the feature extraction method to calculate the Value at Risk of a simple Treasury bond portfolio. These tools are fundamental for understanding more advanced financial theories.

References

- Bjerring, Thomas T. "The Yield Curve and Its Components." *Github*, 16 October 2019, <https://bjerring.github.io/bonds/2019/10/16/the-yield-curve-and-its-components.html>.
- Oprea, Andreea. "The Use of Principal Component Analysis (PCA) in Building Yield Curve Scenarios and Identifying Relative-Value Trading Opportunities on the Romanian Government Bond Market." *Journal of Risk and Financial Management*, vol. 15, no. 6, 2022. <https://www.mdpi.com/1911-8074/15/6/247>.